

Prague University of Economics and Business
Faculty of Informatics and Statistics



An overview and comparison of the Essential Video Coding standard

BACHELOR THESIS

Study program: Applied Informatics

Author: Gheorghe Vornicescu

Supervisor: Bc. et Bc. Jaroslav Svoboda, MSc

Prague, May 2025

Acknowledgements

I would like to thank my thesis supervisor, Bc. et Bc. Jaroslav Svoboda, MSc et MSc, for provided help in writing this thesis.

Abstract

This thesis provides a comprehensive overview and comparative analysis of the MPEG-5 Essential Video Coding (EVC) standard. After tracing the historical evolution of video coding standards and overviewing the technical foundations and toolset of EVC, the work evaluates its encoding efficiency and computational complexity. Objective quality assessments were conducted on three representative test sequences (Sintel, Crowd Run, Park Joy) using VMAF, MS-SSIM, and PSNR-HVS-M metrics. EVC's performance — via its XEVE main and baseline profiles — was compared against other codecs including libaom AV1 (2019), SVT-AV1 (preset 4), libvpx-VP9, x265 (HEVC), and VVenC (VVC). The results demonstrated that EVC's main profile provides competitive visual quality, particularly effective at low and medium bitrates, effectively positioning itself between royalty-free next-generation codecs and those hindered by patents. Additionally, the baseline profile offers a viable alternative characterized by significantly reduced computational demands, albeit accompanied by a compromise in visual quality. Computational complexity was quantified by analyzing encoding time and peak memory usage, with findings showing consistently low memory usage at all bitrates, low encoding time for baseline profile and high encoding time for main profile.

Keywords

Video coding, EVC, standardization, video compression

Abstrakt

Tato práce poskytuje komplexní přehled a srovnávací analýzu standardu MPEG-5 Essential Video Coding (EVC). Po uvedení historického vývoje standardů kódování videa a technických základů EVC práce hodnotí efektivitu komprimace a výpočetní náročnost tohoto standardu. Objektivní kvalita byla testována na třech reprezentativních testovacích sekvencích (Sintel, Crowd Run, Park Joy) pomocí metrik VMAF, MS-SSIM a PSNR-HVS-M. Výkon EVC byl porovnán v rámci hlavního a základního profilu XEVE vůči kodekům jako libaom AV1 (2019), SVT-AV1 (preset 4), libvpx-VP9, x265 (HEVC) a VVenC (VVC). Výsledky ukázaly, že hlavní profil EVC poskytuje konkurenceschopnou vizuální kvalitu, zejména při nízkém a středním bitrate, čímž se účinně řadí mezi kodeky nové generace, které jsou bez licenčních poplatků, a kodeky, jejichž použití brání patenty. Základní profil navíc nabízí alternativu vyznačující se výrazně nižšími výpočetními nároky, i když doprovázenou kompromisem ve vizuální kvalitě. Výpočetní náročnost byla kvantifikována analýzou doby kódování a maximálního využití paměti. Zjištěné výsledky ukazují konzistentně nízkou spotřebu paměti při každé hodnotě bitrate, nízkou dobu kódování pro základní profil a vysokou dobu kódování pro hlavní profil.

Klíčová slova

Kódování videa, EVC, standardizace, komprimace videa

Contents

Introduction	9
1 History of video coding standards	11
1.1 H.120	11
1.2 H.261	11
1.3 MPEG-1	12
1.4 MPEG-2 / H.262	13
1.5 H.263	13
1.6 H.264 / AVC	14
1.7 H.265 / HEVC	15
1.8 AV1	17
1.9 H.266 / VVC	18
2 Development timeline of MPEG-5 EVC	20
2.1 Inception (2017-2018)	20
2.2 Call for proposals and selection (2018-2019)	20
2.3 Standard development (2019)	21
2.4 Final standardization (2020)	21
2.5 Post-standardization	21
3 The principles of EVC	23
3.1 Technical Foundations	23
3.2 Profile Structure	24
3.3 EVC Levels and Constraints	26
3.4 Coding Tools and Techniques	27
3.4.1 Intra Prediction	27
3.4.2 Inter Prediction	28
3.4.3 Transform and Quantization Methods	28
3.4.4 Entropy Coding	29
4 Methods of objective video quality comparison	31
4.1 PSNR-HVS-M	31
4.2 MS-SSIM	32
4.3 VMAF	33
5 Objective quality comparison	34
5.1 Hardware and Software Parameters	34
5.1.1 Tested video sequencing	34
5.1.2 Encoder Parameters	35
5.1.3 Commands Used	35
5.1.4 Other Codecs	36

5.2	Quality Comparison	36
5.2.1	VMAF metric	36
5.2.2	MS-SSIM Metric	39
5.2.3	PSNR-HVS-M Metric	41
5.3	Encoding Time and Memory Usage	43
	Conclusion	47
	References	48
	A Quality testing data	53
	B Time and memory usage	57

List of Figures

1	MPEG Group of Pictures [3]	12
2	Picture, slice, CTU, CU [17].	16
3	H.264/H.265 block subdivision comparison [4].	16
4	Wavefront Parallel Processing. Each row represents a thread. Grey squares represent previously processed CTUs, whereas blue represent currently processed CTUs [13].	17
5	CTU split at picture boundaries; figure on the left: forced HEVC-like quadtree split at picture boundaries; figure on the right: forced split using quadtree or binary-ternary split at picture boundaries [16].	18
6	Block diagram of the EVC Codec [50].	23
7	Results of VMAF tests performed on "Sintel" sequence. Chart created by author using data from own testing,[45] and [35]	37
8	Results of VMAF tests performed on "Crowd Run" sequence. Chart created by author using data from own testing, [45] and [35]	38
9	Results of VMAF tests performed on "Park Joy" sequence. Chart created by author using data from own testing,[45] and [35]	39
10	Results of MS-SSIM tests performed on "Sintel" sequence. Chart created by author using data from own testing,[45] and [35]	40
11	Results of MS-SSIM tests performed on "Crowd Run" sequence. Chart created by author using data from own testing,[45] and [35]	40
12	Results of MS-SSIM tests performed on "Park Joy" sequence. Chart created by author using data from own testing, [45] and [35]	41
13	Results of PSNR-HVS-M tests performed on "Sintel" sequence. Chart created by author using data from own testing, [45] and [35]	42
14	Results of PSNR-HVS-M tests performed on "Crowd Run" sequence. Chart created by author using data from own testing and [35]	43
15	Results of PSNR-HVS-M tests performed on "Park Joy" sequence. Chart created by author using data from own testing and [35]	43
16	Encoding time of selected sequences using baseline profile of xeve. Chart created by author.	44
17	Encoding time of selected sequences using baseline profile of xeve. Chart created by author.	45
18	Maximum RAM used encoding selected sequences using baseline profile of xeve. Chart created by author.	46
19	Maximum RAM used encoding selected sequences using baseline profile of xeve. Chart created by author.	46

List of abbreviations

CCITT Consultative Committee for International Telephony and Telegraphy	DCT Discrete Cosine Transform
ITU-T International Telecommunication Union Telecommunication Standardization Sector	DPCM Differential Pulse-Code Modulation
NTSC National Television System(s) Committee	GOP Group Of Pictures
PAL Phase Alternating Line	VCEG Video Coding Experts Group
MPEG Moving Picture Experts Group	JVT Joint Video Team
EVC Essential Video Coding	CABAC Context-Adaptive Binary Arithmetic Coding
LCEVC Low Complexity Enhancement Video Coding	CAVLC Context-adaptive variable-length coding
XEVE eXtra-fast Essential Video Encoder	CTU Coding Tree Unit
XEVD eXtra-fast Essential Video Decoder	CB Coding Block
	WPP Wavefront Parallel Processing
	AOMedia Alliance for Open Media
	JVET Joint Video Experts Team
	BT Binary/Ternary Tree

Introduction

By 2022, IP video traffic was projected to represent approximately 82 percent of all global IP traffic, up from 75 percent in 2017, driven primarily by the rapid adoption of high-definition and ultra-HD streaming services, as well as the rise in mobile video consumption [6].

Video coding standards have continuously evolved to meet these demands. The MPEG-1 and MPEG-2 standards of the 1990s enabled early digital and broadcast video, while the 2003 H.264/AVC codec achieved about a 50 percent bitrate reduction over MPEG-2 for comparable quality. HEVC/H.265 halved the bitrate relative to H.264/AVC, albeit at the cost of significantly increased encoding complexity in some configurations [31]. Despite its technical gains, HEVC's complex patent pools and licensing costs have hindered its universal adoption, prompting the development of alternative codecs such as AV1. With this context, the Moving Picture Experts Group started working on the next generation of video standards to further improve efficiency while addressing the business realities that affect adoption. In 2018, MPEG launched two parallel projects, Versatile Video Coding (VVC) and Essential Video Coding (EVC), with the aim of significantly increasing compression beyond HEVC and avoiding the roadblocks that hindered HEVC adoption. These efforts culminated in 2020 with the completion of both standards. MPEG-5 EVC aims to deliver coding gains approaching those of the state-of-the-art, but with a simplified patent licensing model and fallback options to ensure no single tool's licensing issue can stall the whole codec. EVC's coding architecture and tools build upon proven techniques alongside new innovations, striking a balance between performance and implementation complexity [7].

Main goal and sub-goals

The main goal of this thesis is to provide a comprehensive overview and comparative analysis of the EVC standard by explaining the historical development and underlying principles of EVC and evaluating its performance against other prevalent video coding standards. To achieve this goal, the thesis is structured around the following sub-goals:

Historical contextualization: To trace the development and evolution of video coding standards leading to the development of MPEG-5 EVC, highlighting key milestones and motivations behind them.

Technical Exploration: To explore the fundamental principles and technical innovations that constitute the EVC standard, examining its architecture and unique features.

Quality Assessment: To conduct an objective comparison of the video quality achieved by EVC against other coding formats, utilizing established metrics like VMAF, PSNR-HVS-M, and MS-SSIM.

Computational Complexity Analysis: To evaluate and compare the computational de-

mands of EVC with alternative standards, evaluating factors such as encoding time and memory usage.

Methodology

To address the aforementioned goals, technical documentation, academic literature, and industry reports will be analyzed and examined to understand the fundamental principles and technical advancements of the MPEG-5 EVC standard. Subsequently, comparative methods will be applied by evaluating the video quality of EVC against other relevant standards using well-established quality metrics to quantify both video quality and computational complexity.

1. History of video coding standards

Video coding standardization has evolved through a series of major innovations over the past several decades. Each new generation of standards achieved higher compression efficiency and enabled new applications, from early videoconferencing to high definition streaming. This section will provide a chronological overview of the most influential video coding standards.

1.1 H.120

H.120 was the first digital video compression standard developed by the CCITT (now ITU-T) in 1984 for low-bitrate videoconferencing applications over early digital networks. The standard was designed for videoconferences over early digital networks and ran at bitrates around 1.5–2 Mb/s (1544 kbit/s for NTSC and 2048 kbit/s for PAL) [42]. H.120 introduced several fundamental ideas in video compression. Notably, it used conditional replenishment, an inter-frame technique where only parts of the image that changed from the previous frame were transmitted. This reduced redundant data and achieved compression ratios on the order of 2:1 to 10:1 in the best cases [14]. Another technique used was the Differential Pulse Code Modulation (DPCM), which transmitted only the difference between the numerical values that represent the information in the video frame. Due to coarse DPCM quantization artifacts such as edge noise and contouring were frequent. With low image quality and poor compression performance, H.120 was never widely adopted commercially and essentially no real-world codecs were built to this standard in later years [42].

1.2 H.261

Approved by the CCITT in 1988, H.261 was the first video coding standard that achieved practical usefulness [42]. This standard established a macroblock, motion estimation and transform coding framework that all subsequent major video standards would follow [32]. H.261 operates in either Common Intermediate format (CIF, 352×288) or Quarter CIF (QCIF, 176×144) resolutions and supports 16x16 luminance and 8x8 chrominance sampling at 4:2:0, focusing on low latency encoding for real-time use over ISDN digital lines, for which it was explicitly designed. Although H.261 itself was used mainly for telephony applications, it proved that a single codec design could be flexible across bitrates and led directly to the development of the first MPEG video standard for consumer media [40].

1.3 MPEG-1

MPEG-1 (standardized as ISO/IEC 11172-2 in 1993, with video design finalized around 1991) was the first MPEG video coding standard and was aimed at VHS quality digital video at about 1–1.5 Mb/s, with a typical format of 352x240 for NTSC, and 352x288 for PAL, although sizes as large as 4k x 4k can be specified by users. Technically, MPEG-1 built on the H.261 architecture, but added new features that significantly improved compression for stored-media applications. One key innovation was the introduction of bidirectional predictive coding - B-frames. In MPEG-1, each frame sequence is divided into one or more Group of Pictures (GOP).

There are three types of frames in a group:

- I-frames (intra-coded still images) which are transmitted only with intraframe coding and are the reference frames for decoding the other frames of the group. Used to start the decoding and play back the received TV signal at almost any time;
- P-frames (predicted from past frames) that are transmitted using inter-frame coding. They are used to predict, with motion compensation, nearest preceding I-frame or P-frame;
- B-frames (bi-directional-coded frames) which are transmitted with inter-frame coding and motion-compensated prediction using the nearest I-frame and P-frame both in front and behind them. B-frames cannot be used for prediction of other frames [3].

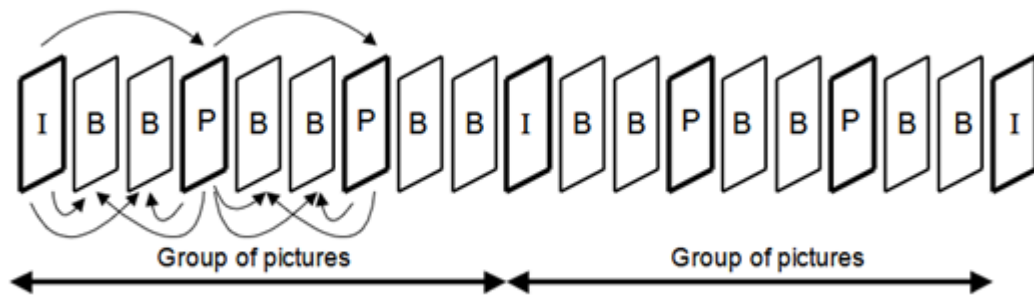


Figure 1: *MPEG Group of Pictures* [3]

Along with slight refinements to DCT quantization and half-pixel motion compensation accuracy, compared to H.261 which had integer-pixel motion vectors, MPEG-1 made it feasible to store and play back video on personal computers and standalone VCD players. In addition to VCD, MPEG-1 video, with a widely known .mpg file format, was common in early web media and software applications. The standard also defined the popular MP3 audio format - MPEG-1 Audio Layer III, further boosting its influence in digital media. By the mid-1990s, MPEG-1 was widely adopted in both hardware, such as CD-i players or PC video capture cards, and software, such as playback libraries on Windows or Mac. Technically, MPEG-1

validated the motion-compensated DCT approach on a larger stage, and many of its concepts were directly carried into later standards. However, it was limited in not supporting interlaced scanning, so all video was treated as progressive frames. This was acceptable for computer and CD-ROM video, but a new standard was soon needed to handle broadcast TV signals and studio-quality video [10].

1.4 MPEG-2 / H.262

MPEG-2 (ITU-T H.262, ISO/IEC 13818-2), finalized in 1994, extended the MPEG-1 approach to support broadcast-quality video and higher resolutions. It became the foundation for digital cable, satellite, and over-the-air TV, as well as the DVD-Video format [43]. Built on MPEG-1's core coding tools, it added crucial features for professional and broadcast use. One major addition was explicit support for interlaced video, the format in which traditional analog TV signals are captured and displayed. MPEG-2 could encode interlaced content efficiently by allowing pictures to be coded as frames or as separate fields, with adaptive field/frame prediction modes [3]. By properly handling interlaced scans, MPEG-2 ensured compatibility with both legacy SDTV (480i/576i) and emerging HDTV formats (1080i).

MPEG-2 quickly became the industry standard for digital television. There was strong backing from the broadcast and electronics industries, which led to its universal adoption in DVB and ATSC standards for digital TV [40]. Even as newer codecs have emerged, MPEG-2 remained in use in many legacy systems and niche applications, having almost a third of the market as late as 2019 [49].

By the mid-1990s, with digital TV and media firmly using MPEG-2, attention returned to low-bitrate video for emerging applications like internet streaming and videophones over plain telephone lines. The ITU-T developed H.263 as an evolution of H.261 for very low bitrates, and around the same time MPEG-4 Visual introduced object-based coding.

1.5 H.263

H.263, approved in 1995 (with enhancements in 1997-2000), was the next-generation ITU-T video codec after H.261, intended for low bitrate communication, such as videoconferencing over dial-up telephone lines and early mobile networks. H.263 was built on the same basic framework of block-based DCT + motion compensation, but included several improvements tailored to low-bandwidth scenarios. Many of these were defined in 24 annexes of the H.263 standard as optional enhancements to the baseline codec. However, even in baseline form, H.263 delivered substantially better compression than H.261 – roughly a twofold improvement in coding efficiency was reported at very low bitrates [44].

In addition to traditional telecom uses, H.263 found broader usage in the early days of

internet video. It was the basis for Flash Video that, in its first versions, used a variant of H.263 for web playback (VoIP-Info, 2016). The mobile industry also embraced H.263 in the early 2000s: the 3GPP and 3GPP2 standards for 3G mobile multimedia messaging specified H.263 for video clips, given its efficiency in low-bandwidth cellular networks [20].

1.6 H.264 / AVC

H.264, also known as AVC (Advanced Video Coding) and MPEG-4 Part 10, was finalized in 2003 as a joint project between the ITU-T Video Coding Experts Group (VCEG) and ISO/IEC MPEG. These groups formed the Joint Video Team (JVT) to develop a codec that would be H.264 built upon the strengths of earlier MPEG-4 and H.26x codecs while introducing innovative techniques to roughly double the compression performance of MPEG-2 [25].

H.264 maintained the fundamental block-based hybrid video coding framework of motion compensation and transform coding but with significant improvements at each stage. Motion prediction was greatly enhanced: frames divided into macroblocks could be further split into variable block sizes down to 8×8 or 4×4 , allowing for finer or coarser motion tracking depending on content complexity. For entropy coding, H.264 introduced the powerful CABAC (Context-Adaptive Binary Arithmetic Coding) algorithm, which compresses binary symbols with context modeling to yield significant bitrate savings at the cost of extra computing load. A lighter-weight CAVLC (Context-Adaptive VLC) method is also provided for lower-complexity encoding [8].

The standard defines multiple profiles to tailor complexity to applications, with the most commonly applied being the Baseline Profile, for low-latency videoconferencing and mobile use, Main Profile for broadcast and storage, and High Profile for Blu-ray and professional HD video. Table 1 shows support for some of the features for these profiles.

Table 1: *Selected feature support. Table created by author using information retrieved from ITU-T Recommendation H.264 V12 [19].*

Feature	Main Profile	Baseline Profile	High Profile
B frames	Yes	No	Yes
CABAC coding	Yes	No	Yes
Bit depth	8	8	8
8×8 and 4×4 adaptive transform	No	No	Yes
Chroma formats	4:2:0	4:2:0	4:2:0

The H.264 standard defines multiple levels, each representing specific constraints tied to the performance requirements of a decoder within a profile. Practically, these levels set

the limits for the maximum allowed video resolution, frame rate, and bit rate supported by the decoding devices. Higher levels support higher resolutions and data rates, while lower levels correspond to lower resolutions, lower maximum bit rates, and reduced memory usage for reference frames storage. Decoders conforming to a particular level must be capable of handling all bit streams encoded at that level and any lower levels [37].

H.264 rapidly became the dominant video codec across industries due to its strong performance and manageable licensing. It was backed by a broad patent pool administered by MPEG LA, which made it relatively straightforward for manufacturers and software vendors to license the technology [8]. By the late 2000s, hardware decoders for H.264 were integrated into PCs, set-top boxes, game consoles, and mobile chipsets, accelerating adoption. This led to H.264 being the default codec for a huge range of applications: broadcast TV began using H.264 for HD channels, the Blu-ray Disc format adopted H.264 High Profile for 1080p video, and major streaming platforms all standardized on H.264 for internet video delivery. The codec struck a balance of high quality and reasonable computational cost that made it feasible even in battery-powered devices by the 2010s. Around 2011, nearly 80% of all video encoded worldwide was reportedly using H.264 [4].

1.7 H.265 / HEVC

H.265, better known as HEVC (High Efficiency Video Coding), was finalized in early 2013 as the official successor to H.264/AVC. It was developed by the Joint Collaborative Team on Video Coding (JCT-VC), a partnership between the same ITU-T VCEG and ISO/IEC MPEG groups, following the completion of H.264. The goal for HEVC was to achieve roughly double the compression efficiency of H.264 to accommodate the rise of ultra-HD formats and next-generation HDTV displays (Sullivan et al., 2012). To achieve this goal, HEVC brought advances in frame divisions. It retained the block-based hybrid codec structure, but instead of using fixed-sized macroblocks, Coding Tree Units (CTUs) were introduced. CTUs can have sizes in the range from 16x16 to 64x64 pixels, compared to static 16x16 in H.264, and are adaptively subdivided into smaller Coding Units using a quadtree structure. It can still subdivide the CTUs further down to 4x4 Coding Blocks (CBs) for the residual data. This improvement allowed for significant efficiency gains [17].

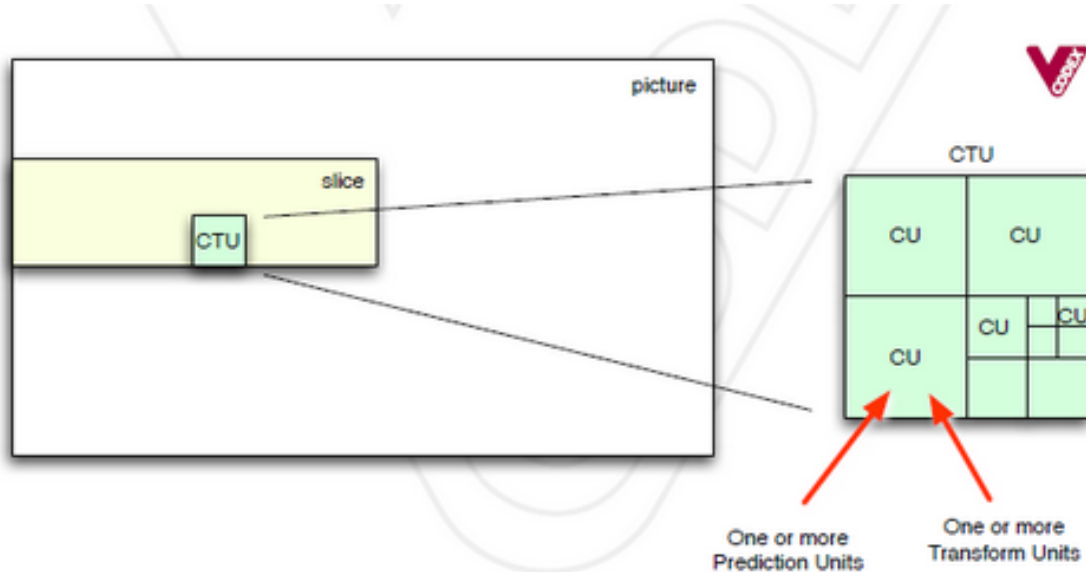


Figure 2: *Picture, slice, CTU, CU [17].*

In [Figure 3](#), a difference can be seen between the H.264 and H.265 subdivisions. It can be noted that in areas with fewer details, bigger blocks are used compared to H.264.



Figure 3: *H.264/H.265 block subdivision comparison [4].*

Another advancement of this standard is the support for parallel processing techniques: Tiles, where a frame is split into multiple rectangular areas for simultaneous decoding, and Wave-front Parallel Processing (WPP) which allows for the parallel decoding of multiple CTUs.

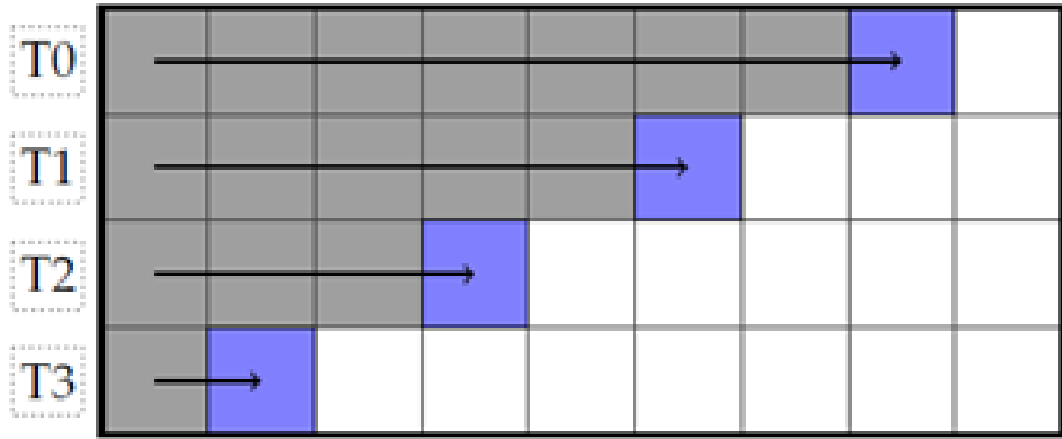


Figure 4: *Wavefront Parallel Processing. Each row represents a thread. Grey squares represent previously processed CTUs, whereas blue represent currently processed CTUs [13].*

1.8 AV1

AV1 is an open, royalty-free video coding format released in 2018 by the Alliance for Open Media (AOMedia), a consortium of tech companies formed in 2015. The creation of AV1 was driven by a collective industry motivation to have a state-of-the-art video codec free from patent licensing fees, as a direct alternative to proprietary standards like HEVC [29]. AV1’s development built upon technologies from several earlier projects: primarily Google’s VP9 codec, and its planned successor VP10, Mozilla/Xiph’s experimental Daala codec, and Cisco’s Thor codec. The best ideas from these were merged into a single design under the AOM umbrella. Development started around 2016, and by March 2018 the AV1 bitstream specification was finalized [38].

AV1 is a traditional block-based hybrid codec at its core, but it introduced many advanced coding tools and optimizations drawn from its parent projects and new research. It uses a structure similar to VP9 in some respects, but with greatly increased flexibility. For example, AV1 operates on superblocks of size 64×64 or 128×128 pixels as the root coding units, which can be recursively subdivided into smaller blocks down to 4×4 based on the image detail [1].

Since its release, AV1 has seen growing adoption, particularly in streaming and web video applications, though not without challenges. One of AV1’s biggest advantages is that many major companies are backing it and have integrated it into their services. For example, YouTube and Netflix – both AOMedia members – have started using AV1 for certain streams. In 2020, Netflix began streaming select titles using AV1 on their Android app, reporting about a 20% efficiency improvement over VP9 for the same quality [12]. YouTube has encoded a large portion of its library in AV1 and serves AV1 videos to devices that support it. On the browser side, Google Chrome, Firefox, Edge, and Opera all added AV1 video decoding

support by 2020

Licensing and patent risk of AV1 has been a topic of concern too. While AOMedia’s members all agreed not to charge royalties and to pool any patents they had on AV1 into a free license, there is always a possibility that third-party patent holders, who are not in AOMedia, might claim AV1 infringes on their patents. In fact, in 2019, a company called Sisvel announced that it had formed a patent pool that claims to cover AV1. This introduced uncertainty: Sisvel offered licenses for AV1 implementers, suggesting that AV1 might not be entirely royalty-free if these patents are valid. However, no major litigation has occurred so far, and many in the industry either dispute the essentiality of those patents or have taken a “wait and see” approach. The perception of being royalty-free remains largely unchanged: Mozilla, Firefox, and others, have no license and ship their products with AV1, and device makers have begun including AV1 codecs widely [21].

1.9 H.266 / VVC

H.266, known as Versatile Video Coding (VVC), is the latest video compression standard from the ITU-T and MPEG collaboration, finalized in mid-2020. It was developed by the Joint Video Experts Team (JVET), which carried forward the work after HEVC, consisting of experts from ITU VCEG and ISO/IEC MPEG – effectively the same bodies behind H.264 and H.265. VVC earned its name because it was designed to handle a wider variety of video scenarios than previous codecs, including not just conventional camera-captured video but also computer-generated content, screen captures, high dynamic range video, and 360-degree VR video [28]. VVC retains the familiar block-based hybrid coding framework of its predecessors but introduces new tools and refinements to achieve more efficiency compared to HEVC. One such feature is a significantly more flexible partitioning scheme. Whereas HEVC used a quadtree to subdivide 64×64 CTUs, VVC uses a multi-type tree (MTT) block partitioning. This allows partitioning units to be split not only in quarters but also in binary splits, horizontal or vertical, and even ternary splits in some cases [16].

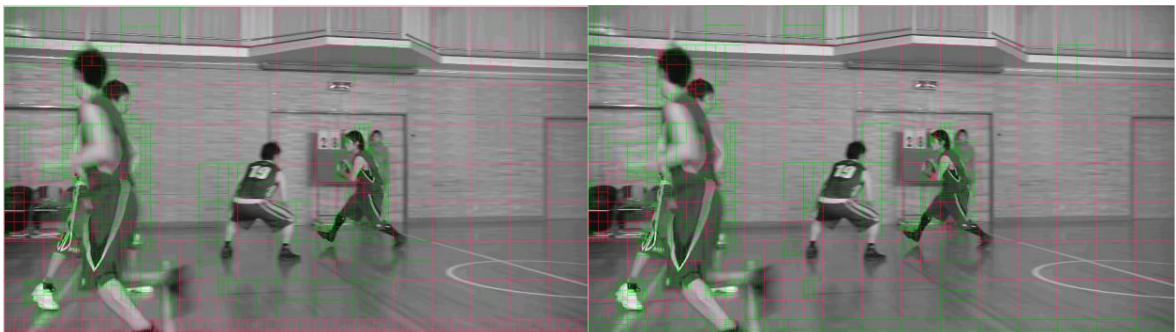


Figure 5: *CTU split at picture boundaries; figure on the left: forced HEVC-like quadtree split at picture boundaries; figure on the right: forced split using quadtree or binary-ternary split at picture boundaries [16].*

In practice, this means a block can be recursively split in a much more adaptive way to fit the content – large uniform areas can remain a single block, while complex areas can be split into many small blocks, with splits of varying orientations. The result is less wasted bits on block boundaries and better modelling of detail. For inter-frame motion prediction, VVC adds advanced capabilities that go beyond traditional translation motion. A major addition is affine motion compensation, which allows use of 2 or 3 motion vectors to model rotation/shear motion of a block with up to six degrees of freedom, useful for objects that undergo more complex movements, for example zooming or rotation [9]. VVC’s profiles at launch include a Main 10 profile, supporting up to 10-bit 4:2:0, which is anticipated to be the most widely used for consumer media, as well as profiles for 4:2:2 and 4:4:4 chroma, plus tools for screen content. Notably, 10-bit coding is a baseline in VVC, reflecting industry move towards higher bit depth. Versatility is also seen in features like scalable and multi-layer coding support, allowing spatial/temporal scalability or multi-view, though these are defined in extensions, and the ability to handle omnidirectional video by supporting projection formats. Overall, VVC’s toolkit is very rich; by combining moderate improvements across many aspects, it achieves a very large cumulative gain. Experiments show bitrate reductions of 30-50% over HEVC at the same quality are common, especially as resolution increases [24]. For instance, for 4K sequences, VVC might save 40% bitrate on average over HEVC for equal VMAF, and even more (up to 50%) for 8K content where its tools shine best [48].

2. Development timeline of MPEG-5 EVC

The creation of MPEG-5 EVC was driven by industry frustrations with licensing hurdles and the desire for a more adoption-friendly codec, in addition to technical efficiency goals. HEVC ended up with at least three separate patent licensing organizations: MPEG LA, HEVC Advance, Velos Media and numerous other patent holders outside those pools [2, 27]. This fragmented patent landscape made it difficult for companies to determine licensing costs and obligations, hindering HEVC’s widespread adoption despite its strong compression performance [2].

2.1 Inception (2017-2018)

The idea for Essential Video Coding arose as MPEG members became concerned about the gap between the technical merits of codecs and their real-world adoption. At MPEG’s 122nd meeting in April 2018, industry representatives explicitly identified the need for a licensing-friendly video codec – one that would have clear and transparent licensing terms and avoid the pitfalls encountered by HEVC. This led MPEG to initiate a new project, which was eventually designated as MPEG-5 Part 1 or EVC. In October 2018, MPEG issued the Requirements for a New Video Coding Standard, which formally outlined the goals for EVC. These requirements emphasized that coding efficiency alone was not enough; the new standard should address real-world use cases and be easy to adopt from both technological and business perspectives, with an upfront plan for licensing [50].

2.2 Call for proposals and selection (2018-2019)

Following the requirements, MPEG announced a Call for Proposals for the EVC standard. By early 2019, two proposal teams had submitted codec designs for consideration. After evaluation, MPEG selected a joint proposal by Samsung, Huawei, and Qualcomm as the basis of the EVC standard [15]. The winning proposal defined a two-tier profile structure – a baseline profile containing only technology either old enough to be patent-expired or offered royalty-free, and a main profile with a small number of new, higher-performance tools. This design addressed the licensing goals: the baseline profile could be used royalty-free, while the main profile’s tools would be individually switchable, allowing implementers to disable any tool that might carry problematic licensing [34].

By early 2019, MPEG had established the foundation of EVC. The project moved forward into a standard development phase with a first Working Draft and an initial Test Model

(ETM 1.0) issued following the proposal selection [11].

2.3 Standard development (2019)

The development of MPEG-5 EVC moved quickly through the MPEG standardization process. An initial Working Draft was completed in January 2019, followed by a Committee Draft in July 2019, and then a Draft International Standard by October 2019 reflecting consensus on the technical content. Throughout 2019, MPEG Video Coding (WG11) experts refined the codec, ensuring that no patent royalties would be required and the new tools of the main profile provided justified gains. During this period, contributors were encouraged to make voluntary licensing declarations to commit to timely and fair licensing terms for any patented tools [26].

2.4 Final standardization (2020)

In April 2020, at MPEG’s 130th meeting, the Essential Video Coding standard was officially finalized. The committee promoted EVC to Final Draft International Standard (FDIS) status, marking the completion of the technical specification ISO/IEC 23094-1. In MPEG’s announcement, it was noted that EVC offers improved compression efficiency compared to prior codecs and that all contributors had committed to announce their licensing terms for EVC’s tools within two years of the standard’s publication [7]. By mid-2020, the standard documents were published by ISO/IEC, and the stage was set for industry adoption.

2.5 Post-standardization

After FDIS approval, work shifted to implementation support: developing reference software, conformance test streams, and reference documentation. These steps, which continued through 2020-2021, were important for industry adoption, ensuring that the EVC bit stream could be validated and that companies had a baseline implementation on which to build on. By mid-2020, MPEG-5 EVC was recognized as one of the three new codecs, along with VVC and LCEVC, being released [7].

With the finalized standard, attention turned to the implementation, compliance, and industry adoption of MPEG-5 EVC. MPEG quickly worked on reference software and EVC conformance suites to facilitate implementers. At the 131st MPEG Meeting, the specification for EVC’s reference software and conformance, or ISO/IEC 23094-4, was promoted to Committee Draft [30].

Outside of MPEG, industry players began to develop their own implementations. In June 2021, Samsung Research released an open-source EVC encoder and decoder – XEVE and

XEVD. These software projects are optimized, ready-to-use implementations of the EVC Baseline and Main profiles, intended to jump-start adoption by offering companies a reference-quality codec they can integrate. In the publication Samsung stated that XEVE provides high-performance encoding that can be deployed in streaming services immediately, while XEVD is lightweight enough for mobile and embedded devices [33].

3. The principles of EVC

This section will cover and explain EVC’s architecture, innovations, profile structure, compatibility, and encoding/decoding processes.

3.1 Technical Foundations

EVC follows the classic hybrid video coding architecture, a design used since the early standards like H.261. Each input frame is divided into blocks that are predicted, transformed, and encoded using both spatial (intra) and temporal (inter) redundancy reduction techniques. EVC’s architecture consists of a video coding layer (VCL), the core compression engine, and a network abstraction layer (NAL) to package the coded data, similar to prior standards such as AVC/H.264 and HEVC/H.265. In the encoder, a prediction of each block is formed either from previously encoded frames (inter-frame prediction) or from neighboring pixels in the same frame (intra-frame prediction). The difference or residual between the original block and its prediction is then transformed using a block transform and quantized to remove insignificant details. The quantized transform coefficients, along with the prediction meta-data, are entropy-coded into a compressed bitstream. The decoder receives this bitstream and reconstructs the video by performing the inverse operations: entropy decoding, inverse quantization, inverse transform, and adding the predictor to recover each block. In-loop filters are applied to the reconstructed frame to reduce compression artifacts before it is stored to predict future frames [52].

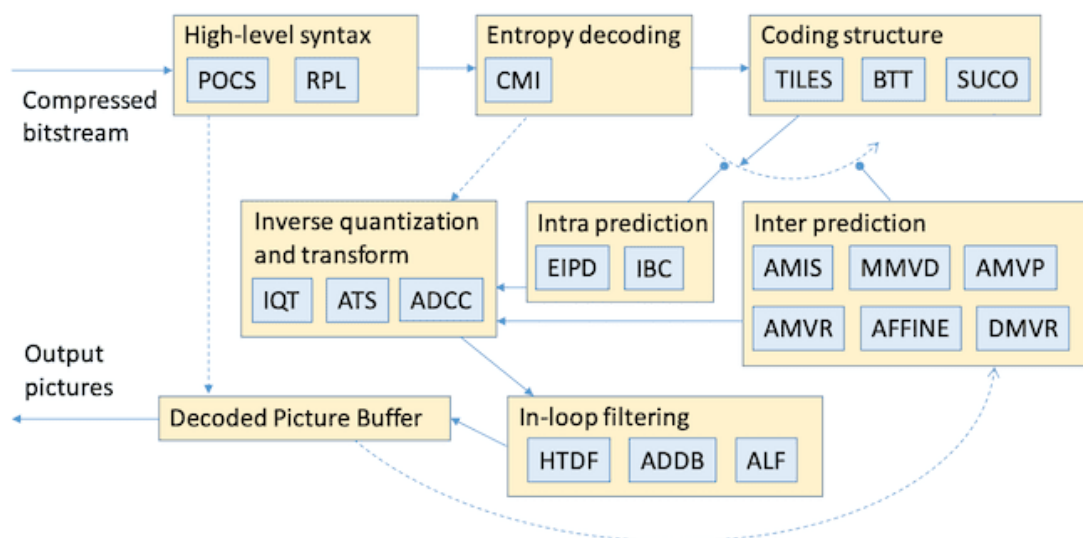


Figure 6: *Block diagram of the EVC Codec [50].*

3.2 Profile Structure

Much like previous MPEG/ITU codecs EVC is separated into different profiles. The standard defines a Baseline profile, a Main profile and a Still-picture profile. The Baseline profile restricts the codec to coding tools that are over 20 years old or submitted as royalty-free, ensuring it can be used without licensing fees. In contrast, the Main profile is a superset that includes additional tools to boost coding performance. Importantly, every tool in Main profile can be switched on or off and replaced by a Baseline tool if needed, which means that an encoder can fall back to Baseline functionality without breaking decoder compatibility [39, 50].

Bitstreams are labeled with a profile indicator, so a Baseline decoder will reject a Main profile bitstream, even if the Main bitstream uses only Baseline tools, whereas a Main decoder can play both Main and Baseline bitstreams. A still-picture profile restricts the bitstream to a single intra-coded frame analogous to a JPEG replacement. This is useful for applications such as image compression or video thumbnails: a Main Still Picture profile would allow the use of all EVC's intra coding tools for maximum compression of a single frame [52].

Table 2 summarizes the core components of the EVC design and how they differ between the Baseline and Main profiles.

Table 2: *Comparison of Baseline and Main Profile in MPEG-5 EVC. Table created by author using information retrieved from [5, 39, 47, 52].*

Coding Stage	Baseline Profile	Main Profile
Block Structure	Quad-tree up to 64×64 ; All CUs must be square.	Quad-tree + Binary/Ternary Tree (BTT) splits for non-square blocks. Split Unit Coding Order (SUCO) allows alternate processing order. Supports larger blocks up to 128×128 .
Intra Prediction	5 modes (DC, horizontal, vertical, diagonal down-left, diagonal down-right). Simple planar interpolation; prediction mode coding uses neighbour modes.	33 modes total: adds 28 directional modes for finer angles. Allows rectangular CUs for intra prediction. Intra Block Copy (IBC) is enabled allowing copying previously decoded blocks from the same frame for prediction.

Continued on next page

Table 2 – continued from previous page

Coding Stage	Baseline Profile (BP)	Main Profile (MP)
Inter Prediction	Traditional translational motion compensation. Uni- and Bi-directional prediction with up to quarter-pixel motion accuracy. Motion vector prediction from up to 4 candidates (3 spatial + 1 temporal + (0,0) vector). Direct temporal mode uses collocated motion. Multiple reference frames with sliding window management.	Adds advanced inter tools: Affine motion for modelling rotation/shear, reusable motion vectors. Flexible reference picture lists (RPL) are signalled in bitstream.
Transforms	DCT-II transform (integer DCT) on residuals, with varying block sizes. No alternative transforms.	Adds transform variety: DCT-II, DST-VII, and DCT-VIII are selectable.
Quantization	Uniform scalar quantization of transform coefficients. Uses a standard linear mapping from transform values to quantized levels.	Improved Quantization (IQT): refined clipping/mapping function to reduce rounding errors. Supports custom quantization matrices (flat by default).
Entropy Coding	Uses a binary arithmetic coder with a fixed probability model. Simplified CABAC-like approach. Uses run-length encoding for zeros.	Context-adaptive binary arithmetic coding with Context Modeling and Initialization (CMI). Advanced Dual Coding of Coefficients (ADCC): 2D run-level coding scheme for efficient coefficient and syntax compression.
In-Loop Filters	H.263 Annex J Deblocking Filter only. Simple block-boundary smoothing. No adaptive filters, so residual blocking noise may remain.	Enhanced filtering: Advanced Deblocking Filter for improved edge smoothing, HTDF applied to luma at high QPs, and Adaptive Loop Filter (ALF) with up to 25 optimized filters on 4×4 blocks.

Continued on next page

Table 2 – continued from previous page

Coding Stage	Baseline Profile (BP)	Main Profile (MP)
Bit Depth / Format	Typically up to 8-bit 4:2:0 color. Focuses on legacy-compatible formats. 10-bit possible but not typical.	Supports 10-bit per component coding. Handles HDR content. Expected to support 4:2:2 and 4:4:4 in future extensions.
Compression Efficiency	50% better than H.264/AVC. Achieves 35–38% bitrate reduction over AVC for 1080p. Lower complexity than HEVC.	25–35% better than HEVC for same video quality. 30% bitrate savings for 4K, 25% for 1080p. Subjective gains up to 50%. Encoding is 5× slower, decoding 1.5× heavier than HEVC.

3.3 EVC Levels and Constraints

EVC standard defines a set of levels that regulate key video stream parameters such as maximum resolution, maximum frame rate, and maximum bit rate. These levels ensure interoperability between encoders and decoders by specifying clear requirements for decoding capability.

Each EVC level is characterized by three main constraints:

- Maximum luma sample rate (in samples per second),
- maximum bit rate (in kilobits per second),
- maximum allowed picture size (resolution) [52].

The officially defined levels and their corresponding limits are summarized in [Table 3](#). These constraints apply equally to both the Baseline and Main profiles.

Table 3: *MPEG-5 EVC level constraints. Table created by author using information retrieved from [18].*

Level	Max luma sample rate (samples/s)	Max bit rate (kbps)	Typical Max Resolution
1	552,960	128	$\sim 640 \times 480$ (VGA)
2	3,686,400	1,500	$\sim 720 \times 576$ (SD)
2.1	7,372,800	3,000	-
3	16,588,800	6,000	$\sim 1280 \times 720$ (HD)
3.1	33,177,600	10,000	-
4	66,846,720	12,000	$\sim 1920 \times 1080$ (Full HD)
4.1	133,693,440	20,000	-
5	267,386,880	25,000	$\sim 3840 \times 2160$ (4K UHD)
5.1	534,773,760	40,000	-
5.2	1,069,547,520	60,000	-
6	1,069,547,520	80,000	$\sim 8192 \times 4320$ (8K UHD)
6.1	2,139,095,040	120,000	-
6.2	4,278,190,080	240,000	-

The maximum luma sample rate determines the total number of pixels a decoder must process each second. The maximum bit rate constraint ensures that the stream’s bandwidth requirements remain within the processing capabilities expected at each level. Resolutions listed are indicative typical values; EVC allows flexibility in picture dimensions provided overall sample rates and bit rates do not exceed level limits.

3.4 Coding Tools and Techniques

3.4.1 Intra Prediction

EVC utilizes intra-frame prediction to use spatial redundancy by predicting blocks from surrounding pixels within the same frame. In the Baseline profile, intra prediction is limited to five modes: average value of the neighbours, vertical, horizontal, diagonal down-left and diagonal down-right. These modes are simple and similar to those found in early standards like H.263. The encoder selects the best mode for each block and signals it, with a mechanism to code it efficiently based on neighbouring block modes. In the Main profile, intra coding is substantially enhanced. Up to 33 prediction directions are available: the original 5 modes plus 28 additional angular directions denoted as Enhanced Intra Prediction Directions (EIPD) covering a wide range of edge orientations for more accurate prediction of textures [50].

Main profile also allows non-square intra-coded blocks, using BTT partitioning, whereas Baseline restricts intra blocks to squares [50]. Another major addition in Main is Intra Block Copy (IBC). IBC treats previously coded regions of the same frame as a reference - essentially

performing a search within the current picture for a matching block. This is highly effective for repetitive patterns or textures such as text or graphics [52].

3.4.2 Inter Prediction

Inter-frame prediction in EVC is built on the traditional motion-compensated prediction framework. The Baseline profile uses a quarter-pixel motion compensation precision, similar to H.264/AVC and HEVC. Each predicted block can use one or two reference frames for uni- or bi-prediction respectively. Baseline uses a median predictor set for motion vectors: up to 5 candidate motion vectors are considered – typically 3 spatial neighbours – from left, above, above-right blocks – and 1 temporal neighbour - collocated block from a reference frame, plus a (0,0) vector for no motion. The best matching predictor is chosen and a motion vector difference (MVD) is signalled if needed. If a candidate predicts the motion of a block well, with no residual and no MVD, it can be coded in Skip mode - no coefficients or MV transmitted. Baseline also supports Direct mode using the collocated motion without sending an MV, inherited from H.263 Annex U. Multiple reference pictures are allowed with a simple sliding window management [5, 52].

The Main profile introduces numerous advanced inter prediction techniques that significantly improve motion prediction efficiency over Baseline profile. One such addition is Affine Motion Compensation. Instead of one motion vector per block, an affine model uses either 2 or 3 motion vectors at different block corners to linearly approximate complex motion [50]. EVC's affine mode supports a four-parameter model and a six-parameter model for more complex motion [26].

Another tool is Advanced Motion Vector Prediction (ADMVP), which expands the candidate pool of motion vectors available to predict the motion of the current block. ADMVP and a related concept called Advanced Merge basically allow the encoder to merge neighbouring blocks that share the same motion without sending an extra vector [52].

The main profile also uses history-based motion vector prediction (HMVP)¹ to make previously used motion vectors available as candidates for new blocks, improving the prediction in cases of repeating motion patterns [26].

3.4.3 Transform and Quantization Methods

After prediction, the encoder compresses the residual signal using transform coding. EVC's Baseline profile uses the well-known block transforms: the Discrete Cosine Transform (DCT) of size 4×4 up to 32×32 , the same DCT-II type used in H.264/AVC and HEVC. The transform

¹sometimes called long-term MV reuse

converts residuals into frequency coefficients, which are then quantized. Baseline profile utilizes a scalar uniform quantizer with a step size governed by the quantization parameter (QP) ranging from 10 to 51. Coefficients are typically scanned in a zig-zag order and quantized by simple division [52].

The Main profile adds improvements in the transform and quantization stage to increase efficiency for different residual characteristics. Notably, Main can choose among multiple transforms: besides the default DCT-II, it includes the Discrete Sine Transform of type VII (DST-7) and an additional DCT of type VIII for certain block sizes. The encoder can perform Adaptive Transform Selection (ATS), deciding which transform (DCT-II or alternative) best compacts the energy of the residual. For example, DST-7 is known to be beneficial for intra predicted residuals on smaller blocks, as it was used in HEVC for 4×4 intra blocks; EVC Main extends this concept by offering more transform choices at larger sizes as well. By selecting the optimal transform basis per block, Main profile achieves improved coding gain especially for directional textures and directional intra residuals. Additionally, Main profile introduces an Improved Quantization (IQT) method. Rather than the standard linear rounding, IQT involves a refined mapping or clipping of coefficients to quantization levels, possibly using a smaller quantization step for low-energy coefficients to preserve detail (as implied by the mention of a different mapping/clipping function). This can reduce distortion for the same bitrate, although with a slight increase in complexity. The combination of adaptive transforms and improved quantization allows the Main profile to retain more detail in compressed video, compared to Baseline. It's worth noting that all transforms in EVC are computed with integer math and the quantization remains scalar and uniform. The additional transforms and quantization tweaks are similar in spirit to those added in newer codecs like VVC, but EVC selected a small, efficient subset to keep complexity manageable [52].

3.4.4 Entropy Coding

Entropy coding in EVC is the final stage that losslessly compresses motion information, modes, and quantized coefficients into the bitstream. The Baseline profile uses a simpler entropy coder to avoid recent patented techniques. Specifically, Baseline EVC uses the binary arithmetic coding algorithm from JPEG's Annex D. This coder uses a fixed set of probability models and state transition rules to encode binary decisions. In practice, it functions similarly to the arithmetic coding in H.263 or the simplified CABAC used in MPEG-4 Part 2. Importantly, Baseline's arithmetic coder does not use adaptive context modelling based on neighbouring symbols beyond what is built into those fixed probability states. For example, coefficient data might be coded with a static context or a very limited context derived from coefficient position, rather than using neighbouring block information. Baseline also relies on classic run-length coding for quantized coefficients: after zig-zag scanning a block's coefficients, it codes runs of zeros and the values of nonzero coefficients, analogous to the method used in MPEG-2 and MPEG-4 Visual [50, 52].

The Main profile replaces and extends the entropy coding with a more efficient and modern

approach. It uses a CABAC-like system with improved context modeling and initialization. In Main, each binary symbol is encoded with an arithmetic coder that updates its probability based on preceding coded data. The context modeling uses neighboring already-decoded symbols to select probability models. For instance, the entropy coder might use the magnitude of adjacent coefficients or the prediction modes of neighboring blocks to choose the probability context for the current coefficient or mode, thereby capturing local statistics. This adaptive approach significantly improves coding efficiency by reducing entropy - higher probability for likely symbols, lower for unlikely ones. The result is a more compact bitstream: the Main profile's entropy coding achieves higher compression than Baseline's coder, especially for high-detail or unpredictable content. However, it comes at the cost of extra computational complexity in both the encoder and decoder, since CABAC involves many probability updates and binary arithmetic operations per symbol. This trade-off is acceptable in the Main profile given its efficiency gains [50, 52].

4. Methods of objective video quality comparison

Objective video quality assessment metrics are algorithms that predict the perceived quality of a video by mathematical analysis, as opposed to subjective methods that rely on human viewers. Over the years, many such metrics have been developed to approximate human visual perception. This chapter will describe three important objective full-reference metrics – PSNR-HVS-M, MS-SSIM, and VMAF – including basic concepts and relevant formulae, as a preface to their usage in later chapters.

4.1 PSNR-HVS-M

Perceptually Weighted Peak Signal-to-Noise Ratio with Masking (PSNR-HVS-M) is an enhanced version of the PSNR metric that was developed in the mid-2000s as researchers sought to improve the PSNR correspondence with perceived quality. PSNR-HVS-M works by modifying the standard mean squared error (MSE) computation in two ways to better reflect what humans see in a distorted image [36].

Conventional PSNR is a widely used image/video quality metric defined via the pixel-wise Mean Squared Error (MSE). Given an original image I and a compressed image K of size $M \times N$, the MSE is:

$$MSE = \frac{1}{MN} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} [I(i,j) - K(i,j)]^2 \quad (4.1)$$

and PSNR (in decibels) is then calculated as:

$$PSNR = 20 \log_{10}(R) - 10 \log_{10}(MSE) \quad (4.2)$$

where R is the maximum possible pixel value (e.g. 255 for 8-bit depth). Higher PSNR in dB indicates the compressed image is more similar to the original.

The computation of PSNR-HVS-M can be summarized as follows: the reference and distorted images are divided into 8×8 blocks and transformed (e.g. via DCT); the difference in each transform coefficient is multiplied by a weight from the CSF to account for contrast sensitivity, and further adjusted according to local masking thresholds. Let $\delta_{PSNR-HVS-M}(i,j)$ represent the perceptually weighted error for pixel (i,j) . The HVS-weighted MSE is then:

$$MSE_{HVS-M} = \frac{1}{MN} \sum_{I=1}^{M/8} \sum_{J=1}^{N/8} \left(\sum_{i=1}^8 \sum_{j=1}^8 (\delta_{PSNR-HVS-M}(i,j))^2 \right) \quad (4.3)$$

where (I, J) indexes the 8×8 block within the image and (i, j) indexes the pixel within that block. Because of the perceptual weighting, the PSNR-HVS-M values often differ from conventional PSNR; for example, distortions like blurring or ringing that are more visible to viewers will yield a lower PSNR-HVS-M than the plain PSNR would indicate. PSNR-HVS-M values are reported in dB like the PSNR [46].

4.2 MS-SSIM

The Structural Similarity Index (SSIM) marked a shift from error-based quality measures to perception-inspired similarity measures. SSIM compares local patterns of pixel intensities between reference and distorted images, focusing on luminance, contrast, and structural similarity. Its success in correlating with subjective quality led to further improvements, notably Multi-Scale SSIM (MS-SSIM), proposed by Wang, Simoncelli and Bovik [51]. MS-SSIM was developed to address the fact that human visual perception is multi-scale: we perceive image structures differently depending on viewing distance or resolution.

MS-SSIM extends the single-scale SSIM by computing the similarity across a range of spatial scales. This is achieved by iteratively low-pass filtering the images and downsampling, thus creating a pyramid of progressively lower resolution versions of the reference and distorted images. At each scale j (where scale 1 is the original full resolution, and scale M is the coarsest), the SSIM components are computed: luminance (l_j), contrast (c_j), and structure (s_j) comparisons between the two images. In MS-SSIM, only the lowest resolution scale M uses the luminance term, while contrast and structure terms are computed at every scale. The rationale is that large-scale luminance differences are sufficiently captured at the coarsest scale, whereas contrast and structural details matter at all scales. The final MS-SSIM index is a weighted combination of these multi-scale comparisons. Formally, if M is the total number of scales, the MS-SSIM between image x and y is given by:

$$MS\text{-}SSIM(x, y) = [l_M(x, y)]^{\alpha_M} \cdot \prod_{j=1}^M [c_j(x, y)]^{\beta_j} [s_j(x, y)]^{\gamma_j} \quad (4.4)$$

where l_M is the luminance comparison at the coarsest scale M , c_j and s_j are the contrast and structure comparisons at scale j , and α_M , β_j , γ_j are exponents applied to adjust the relative importance of these components. Typically, $\alpha_M = \beta_j = \gamma_j = 1$ for simplicity, or the values are tuned empirically. Comparisons l, c, s on each scale are calculated in a manner similar to SSIM: for example, $s_j(x, y) = \frac{2\sigma_{xy,j} + C_3}{\sigma_{x,j}^2 + \sigma_{y,j}^2 + C_3}$, measuring how well the local variations of the reference and distorted images align after filtering to scale j . By combining information across scales, MS-SSIM captures both fine details and coarse structural distortions in a single quality score, which ranges from 0 to 1, with 1 meaning perfect similarity [51].

4.3 VMAF

Video Multi-method Assessment Fusion (VMAF) is a modern objective video quality metric developed by Netflix around 2015–2016 as a response to the limitations of existing metrics in predicting streaming video quality.

VMAF’s approach is fundamentally different from single-formula metrics. It combines multiple metrics through a trained model to output a single quality score. In the initial design, three primary full-reference features were used:

- Visual Information Fidelity (VIF): a metric that measures the amount of information preserved in the distorted video relative to the reference, based on statistics from the natural scene and information theory. VIF is sensitive to blur and loss of texture, as it essentially quantifies how much original signal can be extracted from the compressed video given the presence of distortion.
- Detail Loss Metric (DLM): a feature designed to capture the amount of high-frequency detail lost in the compression process. DLM penalizes smoothing or blurring of fine details. It works by comparing edge and texture details between original and compressed frames, resulting in a lower score when detail is lost.
- Temporal Information (TI) or motion coherence features: metrics that capture temporal aspects of quality, such as the stability of frames over time or the presence of motion artifacts. For example, a temporal feature might measure frame-to-frame differences detecting jerkiness or flicker introduced by compression.

These features are computed for the video sequence and then combined using a machine learning model. Originally, Netflix used a Support Vector Machine (SVM) regressor to fuse the metrics. The SVM takes the VIF, DLM and TI feature values and outputs a weighted sum that is trained to best match subjective Mean Opinion Scores of human viewers. In other words, during development, a dataset of video clips with known human quality scores was used to find the optimal weights and non-linear function f such that:

$$VMAF_{pred} = f(VIF, DLM, TI) \quad (4.5)$$

where f is learned to maximize correlation with the true perceived quality. The result is a model that produces a quality score typically on a scale of 0 to 100, where 100 means that the compressed video is indistinguishable from the original and lower values indicate worse quality, with scores in the 20s or 30s often corresponding to very poor quality. Importantly, VMAF is extensible – the model can be retrained with additional features if new metrics emerge. However, the core concept remains a fusion of metrics approach [41, 53].

5. Objective quality comparison

This chapter presents a comparative evaluation of the video quality of the EVC codec against four other major video codecs: H.265/HEVC, AV1, H.266/VVC and VP9.

5.1 Hardware and Software Parameters

Encoding and decoding were performed on a system with the following specifications:

- OS: Arch Linux 6.14.4
- CPU: Ryzen 5 2600 3.40 GHz
- GPU: NVIDIA GeForce RTX 2060 Super
- RAM: 16 GB DDR4 @ 3200 MHz
- ffmpeg version n7.1.1
- xeve version 0.5.0
- xevd version 0.5.0
- vqmt version 1.1

XEVE and XEVD are open source tools that comply with the MPEG-5 EVC standard used to encode and decode video.

5.1.1 Tested video sequencing

For this comparison, three video sequences were selected - "Crowd Run", "Park Joy" and "Sintel" - matching those used in works by Pydych [45] and Hovorka [35], to allow for quality testing against previous results.

Table 4: *Summary of video sequences used for testing*

Name	Resolution	Framerate (fps)	Frame count
Sintel	4096 × 1744	24	240
Crowd Run	3840 × 2160	50	500
Park Joy	3840 × 2160	50	500

All videos are 10 seconds long, with the exception of "Sintel", which was cut down to 10 seconds starting from 04:07.000.

5.1.2 Encoder Parameters

Testing was performed on both the Main and Baseline profile of the XEVE encoder, utilizing 8 threads of a CPU - maximum allowed by the encoder.

XEVE provides the following presets:

- fast,
- medium,
- slow,
- placebo.

For all sequences, the "slow" preset was chosen, as it produced encoded videos with better quality than faster presets, while cutting down encoding time with minimal losses compared to placebo.

To correspond with the data from previous works, the following bitrate values were chosen: 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000, 12000, 14000, 16000, 18000, 20000, 22000, 24000. However, for testing of XEVE's baseline profile, bitrate of 2000 was dismissed, due to inability of encoder to keep to the target bitrate.

5.1.3 Commands Used

Following commands were used for the whole testing process and were incorporated into the script.

Encoding with XEVE:

```
xeve_app -i "$INPUT_Y4M" -w $WIDTH -h $HEIGHT -z $FRAMERATE \  
--threads $THREADS --preset $PRESET --profile $PROFILE --bitrate $BITRATE
```

Decoding with XEVD:

```
xevd_app -i "$EVC_OUT" -m $THREADS -o "$Y4M_OUT"
```

VMAF scoring:

```
ffmpeg -i "$INPUT_Y4M" -i "Y4M_OUT" -lavfi libvmaf -f null - 2> "$VMAF_LOG"
```

PSNR-HVS-M and MS-SSIM scoring:

```
vqmt "$INPUT_Y4M" "Y4M_OUT" $HEIGHT $WIDTH $FRAMES 1 result PSNRHVSM MSSSIM
```

- \$WIDTH: width of a video.
- \$HEIGHT: height of a video.
- \$FRAMERATE: framerate of a video.
- \$PROFILE: encoder profile.
- \$FRAMES: number of frames in a video.
- \$THREADS: CPU threads used to encode and decode the video.
- \$PRESET: encode preset.
- \$BITRATE: target bitrate.
- \$VMAF_LOG: path to folder containing logs produced by VMAF.
- \$EVC_OUT: video encoded with xeve.
- \$INPUT_Y4M: raw video sequence.
- \$Y4M_OUT: raw video decoded from \$EVC_OUT with xevd.

5.1.4 Other Codecs

Data collected from the tests will be compared to the data obtained using the following codecs:

- libaom AV1 (2019) - An early version of the AV1 codec from 2019.
- SVT-AV1 (preset 4) - A version of the SVT-AV1 codec configured with preset 4, offering a balanced trade-off between encoding speed and video quality.
- libvp9-vp9 - A predecessor to AV1 and a contemporary alternative to HEVC, widely used in web video streaming.
- x265 - An open-source implementation of the H.265/HEVC standard, offering higher compression efficiency than H.264.
- VVenC - An open-source implementation of the H.266/VVC standard.

5.2 Quality Comparison

5.2.1 VMAF metric

Looking at [Figure 7](#), VVenC clearly outperforms all other codecs at bitrates up to 6000, especially at lower bitrates. Mid-range bitrates reveal improved convergence among codecs. Here, libaom_av1_2019 and x265 show strong performance, rapidly rising to high VMAF scores around 97-98 by 8000 kbit/s. Libvp9_vp9, xeve_main, and xeve_baseline display competitive yet slightly inferior quality, stabilizing around a VMAF of 96-97 within the same bitrate range. Usage of additional bitrate reaches diminishing returns roughly at 20000 kbit/s, where all codecs hit the plateau.

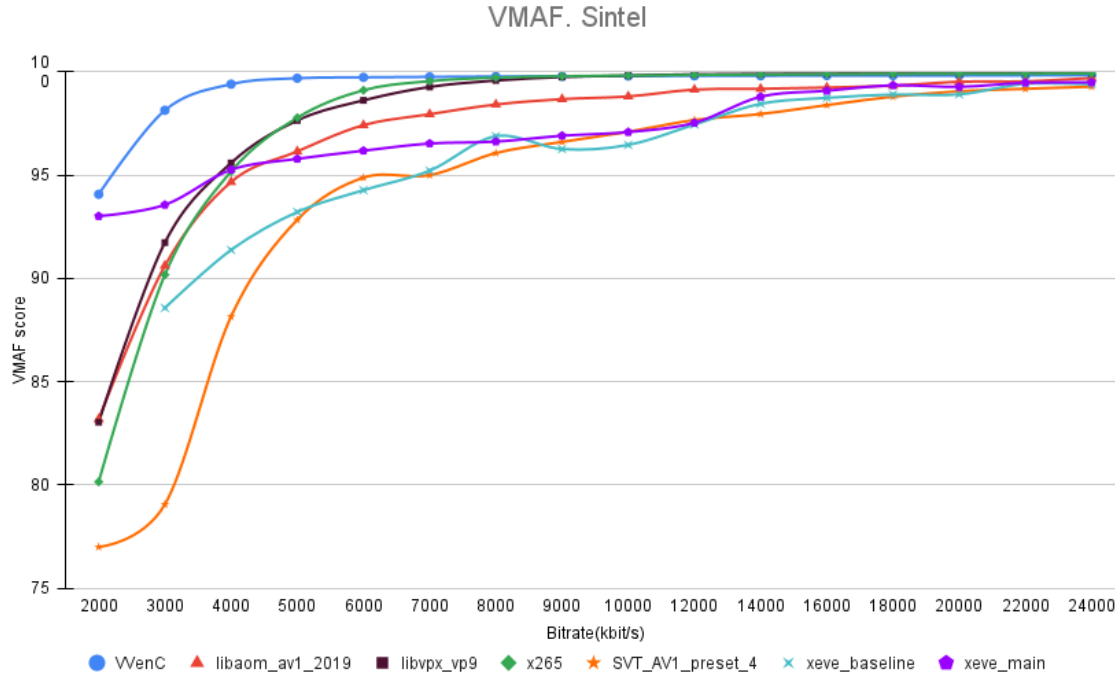


Figure 7: Results of VMAF tests performed on "Sintel" sequence. Chart created by author using data from own testing, [45] and [35]

At Figure 8 VVenC again outperforms other codecs on the whole range of bitrates. At higher bitrates, performance differentiation persists, although reduced, with VVenC still leading substantially. xeve_main follows closely and barely reaches VVenC's level. At 24000 kbit/s, where VVenC gains a significant boost to the VMAF score, xeve_main starts to level. The remaining codecs struggle significantly at lower bitrates and display minimal performance distinctions among themselves after the 7000 kbit/s mark. Perhaps, with different tools selected, xeve_main can reach and even outperform VVenC at a given bitrate. It is important to note that "Crowd Run" is filmed on camera, as opposed to "Sintel", which is entirely computer generated. In the graphs, this is reflected in the linear growth of the VMAF score in Figure 8 compared to a jump in the score in Figure 7.

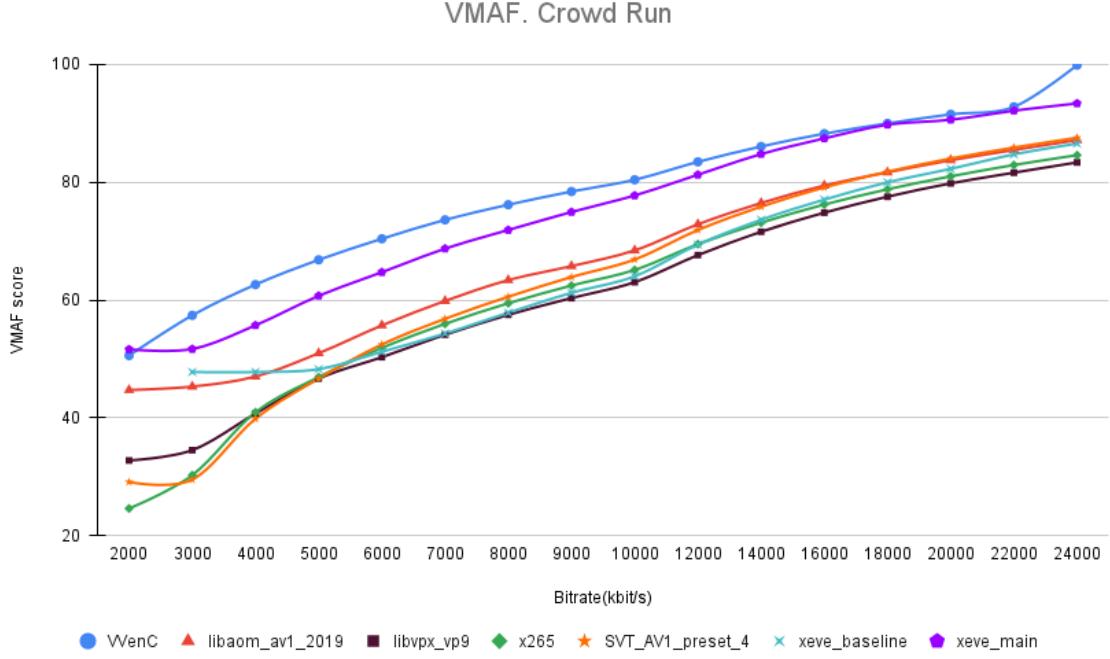


Figure 8: Results of VMAF tests performed on "Crowd Run" sequence. Chart created by author using data from own testing, [45] and [35]

Figure 9 shows curve shapes similar to Figure 8, due to the nature of "Park Joy" being camera-shot. VVenC consistently leads across all bitrate ranges. xeve_main maintains a significant gap to VVenC but outperforms all others. It starts near 52 VMAF at 4000 kbit/s, rises steadily to 78 at 16000 kbit/s, and achieves about 90 at the highest bitrate. xeve_baseline starts stronger than other codecs, and for midrange bitrates performs worse only when compared to xeve_main, VVenC and SVT_AV1_preset_4. However, at 18000 kbit/s performance drops, and, at highest bitrate performs the worst.

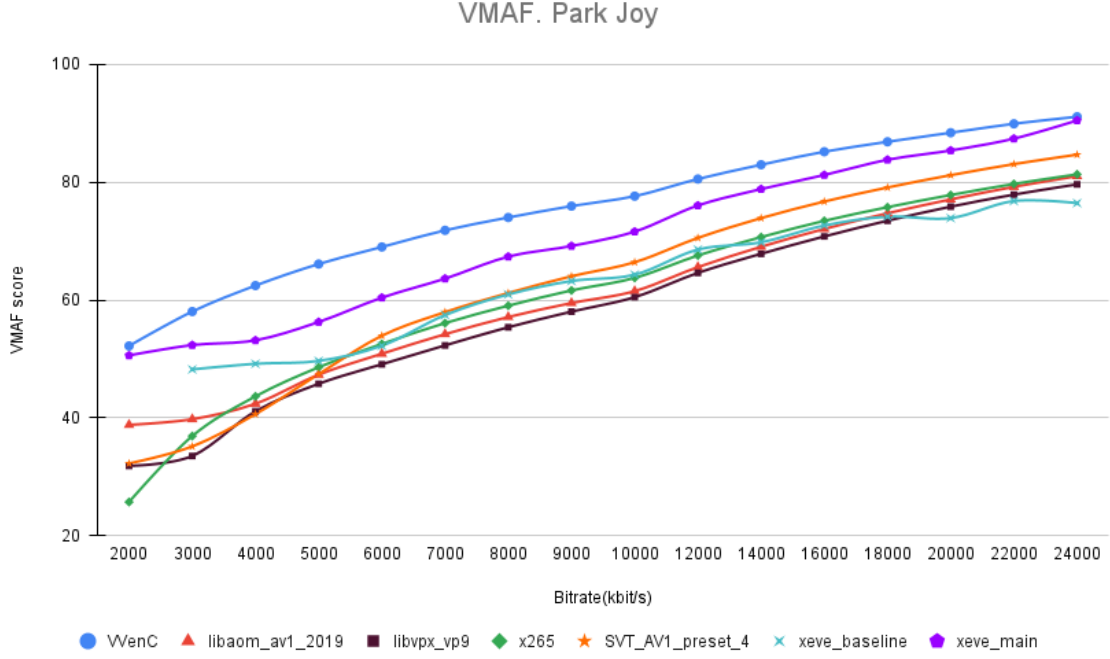


Figure 9: Results of VMAF tests performed on "Park Joy" sequence. Chart created by author using data from own testing, [45] and [35]

5.2.2 MS-SSIM Metric

In Figure 10, with the exception of 2000 kbit/s, VVenC again holds the lead consistently across the bitrate values. xeve_main stagnates at lower bitrates and moves slightly upward starting from the mid point, approaching VVenC only in 20000 to 24000 kbit/s range. On the other hand, xeve_base performs badly, up until 20000 kbit/s mark where it matches the score of libaom_av1_2019 and outperforms x265 and SVT_AV1_preset_4.

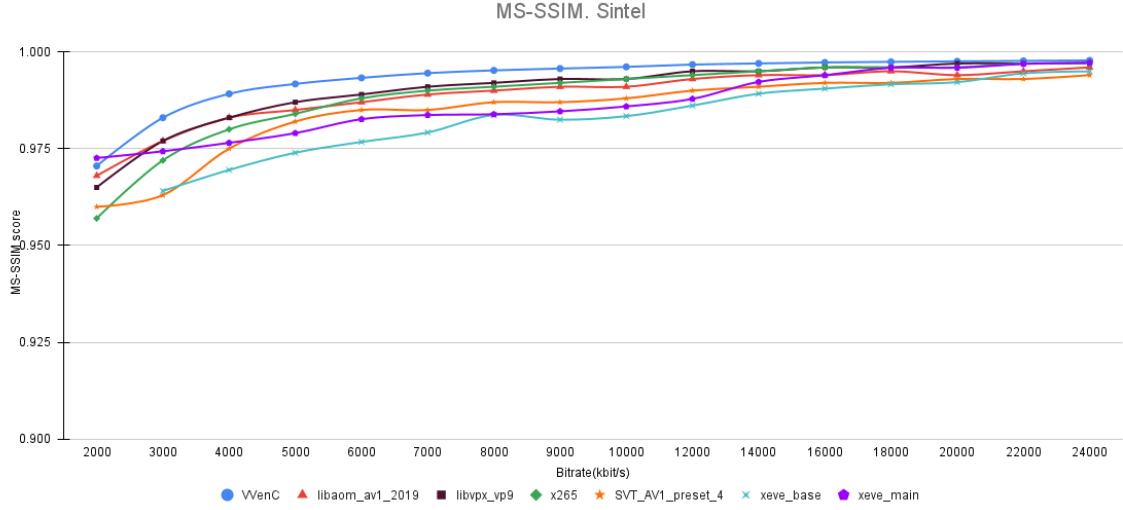


Figure 10: Results of MS-SSIM tests performed on "Sintel" sequence. Chart created by author using data from own testing,[\[45\]](#) and [\[35\]](#)

Result of test seen in [Figure 11](#) and [Figure 12](#) are puzzling. xeve_base consistently performs worse than majority of the codecs presented, even at the highest bitrates. xeve_main closely follows x265 and libvpx_vp9 at lower bitrates and gradually gains more score approaching higher bitrates. When encoding a scene with less complexity, as seen in [Figure 12](#), libaom_av1_2019 performs at 0.872 MS-SSIM score even at 2000 kbit/s; almost a 0.05 difference with xeve_main. At 24000 kbit/s, the score difference is negligible.

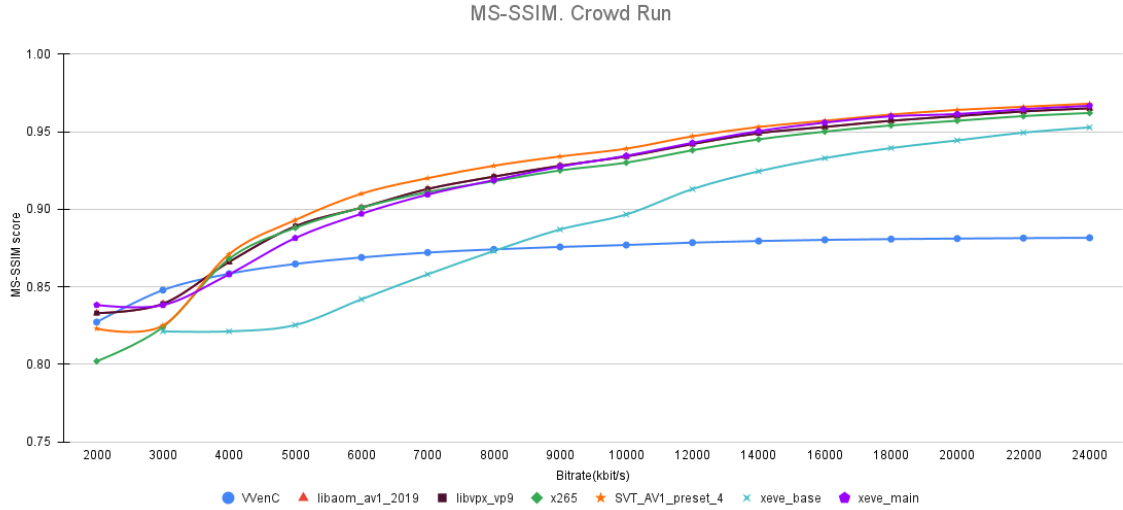


Figure 11: Results of MS-SSIM tests performed on "Crowd Run" sequence. Chart created by author using data from own testing,[\[45\]](#) and [\[35\]](#)

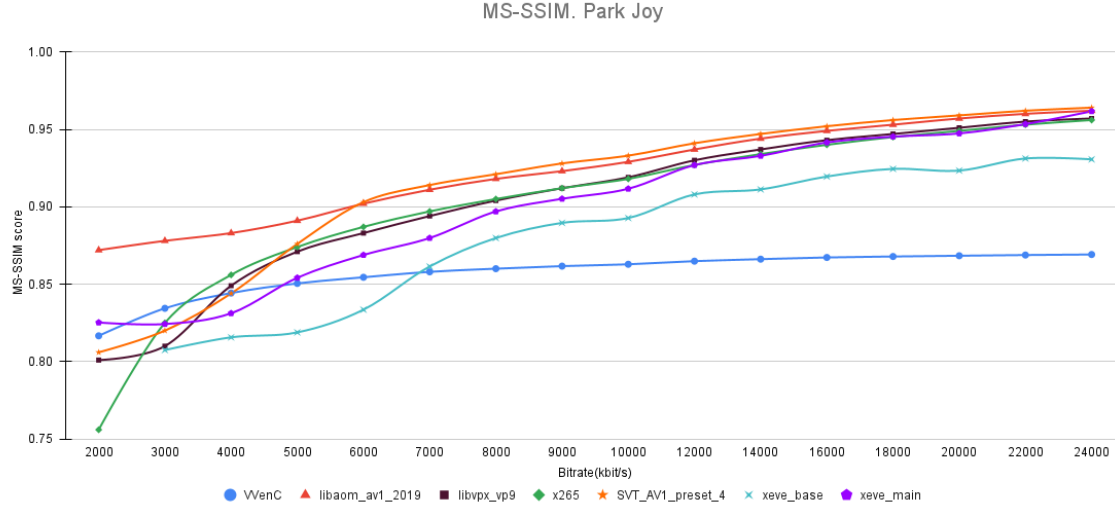


Figure 12: Results of MS-SSIM tests performed on "Park Joy" sequence. Chart created by author using data from own testing, [45] and [35]

5.2.3 PSNR-HVS-M Metric

On the computer-generated "Sintel", the xeve_main begins slightly ahead of libaom_AV1_2019 and SVT_AV1_preset_4 and maintains a narrow lead over the other royalty-free codecs up to about 5000 kbps. As bitrate increases into the 5000–12 000 kbit/s range, VVenC overtakes, yet at the highest rates xeve_main once again ascends to the top with a PSNR-HVS-M score of 50 dB. The baseline profile lags by roughly 2–3 dB throughout, reaching only about 40 dB at 6000 kbit/s and 48 dB at 24000 kbit/s, while libvpx_VP9, x265, and SVT_AV1_preset_4 consistently occupy the middle tier, typically 0.5–1.5 dB below main profile of xeve.

In the camera-captured "Crowd Run" sequence, the curves cluster more tightly. At 2000 kbit/s, libaom_AV1_2019 leads at 24.5 dB, with xeve_main close behind at 23.5 dB and the baseline profile at 22.5 dB. By 6000 kbps, all codecs converge in a narrow band.

"Park Joy" exhibits a similar ordering: libaom_AV1_2019 leads at low bitrates, however SVT_AV1_preset_4 takes the lead starting from 8000 kbit/s. All profiles climb steadily as the bitrate increases, and by 24 000 kbit/s XEVE main, VP9, and x265 coalesce around 30.5 dB, while baseline profile ox XEVE is about 2 dB lower.

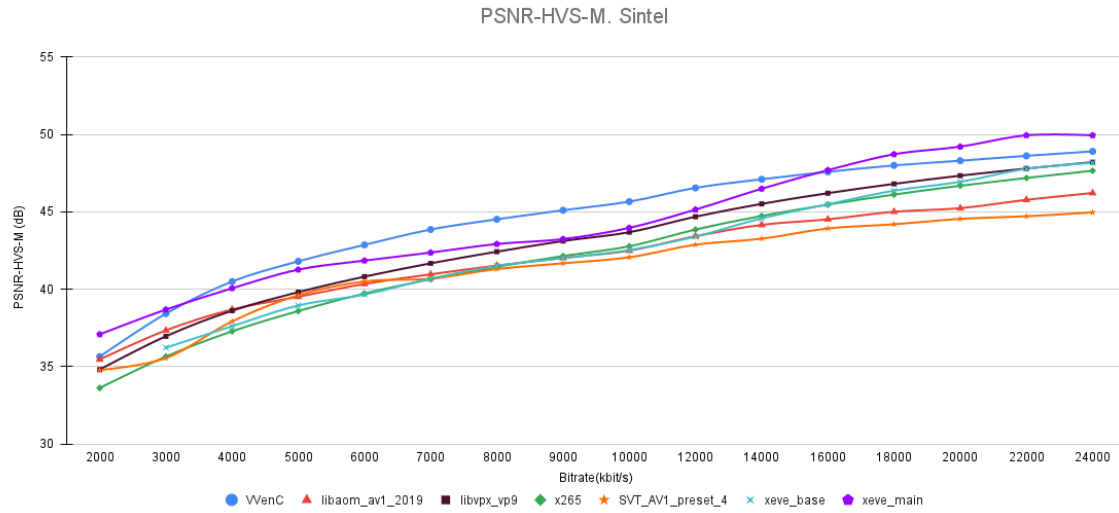


Figure 13: Results of PSNR-HVS-M tests performed on "Sintel" sequence. Chart created by author using data from own testing, [45] and [35]

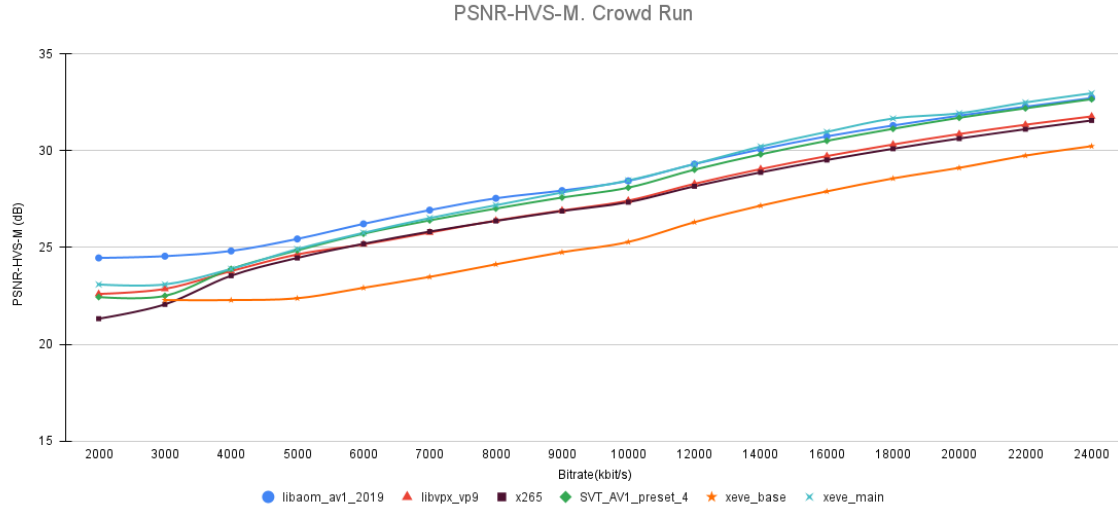


Figure 14: Results of PSNR-HVS-M tests performed on "Crowd Run" sequence. Chart created by author using data from own testing and [35]

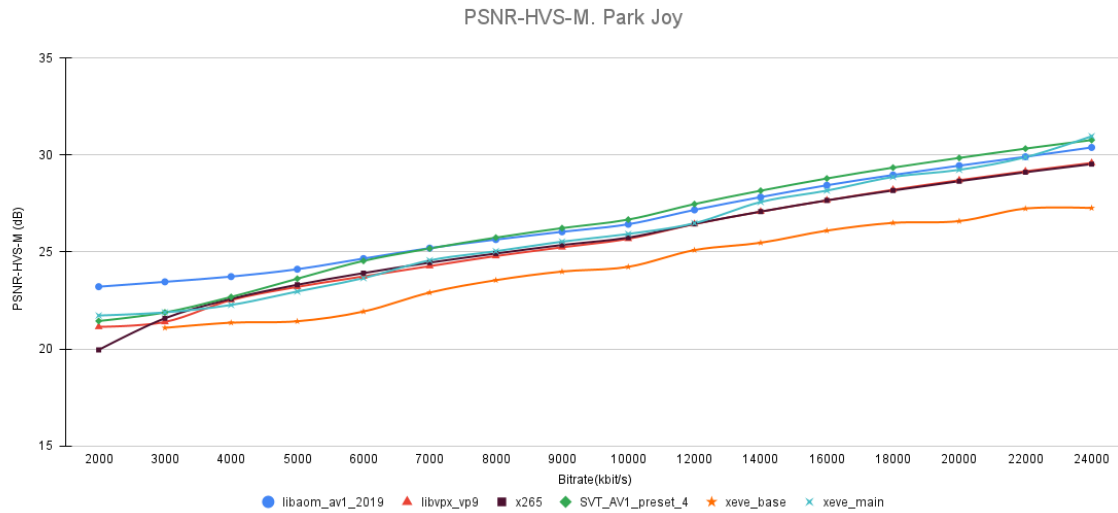


Figure 15: Results of PSNR-HVS-M tests performed on "Park Joy" sequence. Chart created by author using data from own testing and [35]

5.3 Encoding Time and Memory Usage

To record the encoding time and amount of memory used by the encoder, the following command was used

```
/usr/bin/time -v -o "$ENC_TIME"
```

where parameter "\$ENC_TIME" is a path to the folder containing the output text files.

Under the baseline profile, encoding times increase moderately and almost linearly with bitrate. For "Sintel", times rise from roughly 520 seconds at 2000 kbit/s to 692 seconds at 24000 kbit/s. "Crowd Run" starts near 825 seconds and climbs to approximately 1030 seconds, while "Park Joy" moves from around 830 seconds to 960 seconds over the same range. This narrow band means that the baseline profile remains relatively lightweight, with sequence complexity exerting only a modest influence on run-time.

In contrast, the main profile incurs an order of magnitude greater encoding cost that accelerates steeply with bitrate. "Sintel" encoding escalates from about 7834 seconds at 2000 kbit/s to 32092 seconds at 24000 kbit/s. "Park Joy" climbs from 12532 seconds to 40663 seconds, and Crowd Run, owing to its higher motion complexity, soars from 12532 seconds at the lowest bitrate to nearly 48887 seconds at the highest.

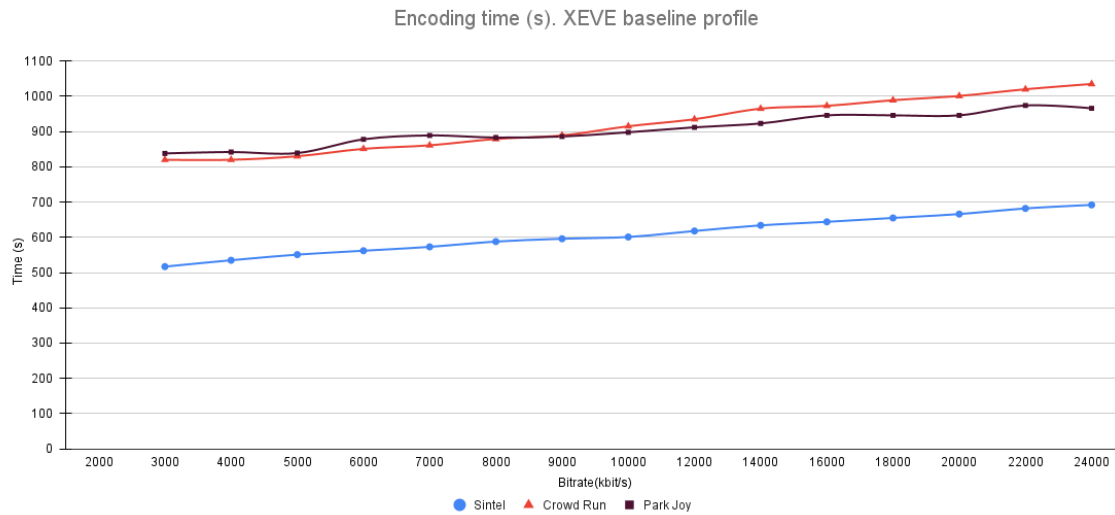


Figure 16: *Encoding time of selected sequences using baseline profile of xeve. Chart created by author.*

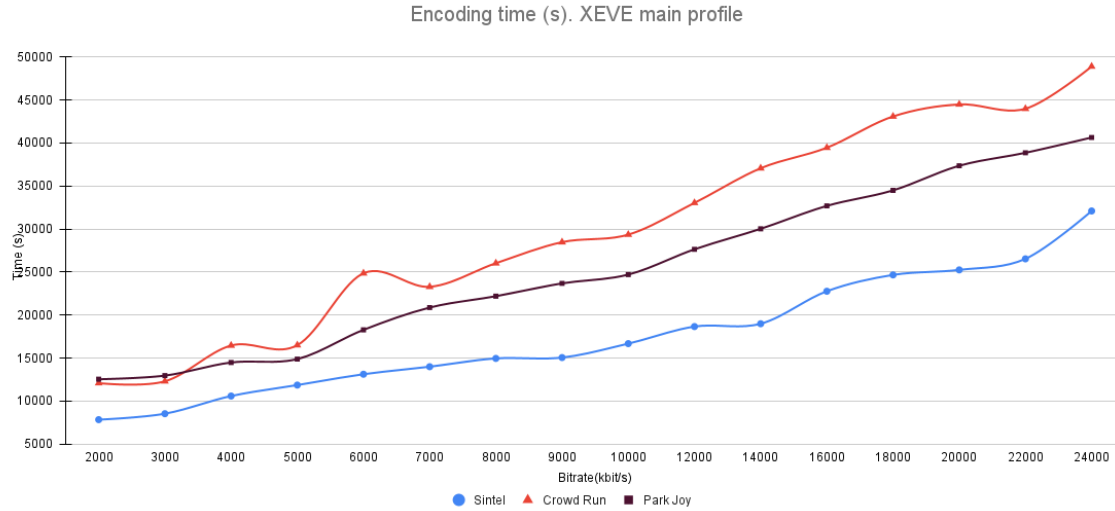


Figure 17: *Encoding time of selected sequences using baseline profile of xeve. Chart created by author.*

As seen in [Figure 18](#), baseline profile uses nearly constant amount of RAM across all bitrates and sequences. For the "Sintel" sequence, peak RAM hovers consistently around 2450 MB from 2000 kbit/s to 24000 kbit/s. "Crowd Run" and "Park Joy" both required slightly more memory - approximately 2600 MB. They show only minor fluctuations of ± 20 MB as bitrate increases, likely due to interference from other processes running on the testing machine.

In contrast, the main profile consumes substantially more RAM and shows notable variability between sequences and over bitrate, as seen in [Figure 19](#). For "Sintel", the main profile allocates roughly 3400 MB of memory consistently across the entire bitrate range. In "Crowd Run", memory climbs progressively from about 2100 MB and peaks near 2750 MB at 18000 kbit/s before dipping and rising again at 22000 kbit/s and 24000 kbit/s respectively, suggesting dynamic activation of high-order tools in motion-intensive segments. "Park Joy" starts near 1950 MB at low bitrates and steadily increases to about 2500 MB by 24000 kbit/s.

Overall, both profiles show significantly lower RAM usage than SVT_AV1_preset4 and VVenC with longer encoding time, while suffering negligible losses in quality, especially in computer-generated footage, and across all sequences when using xeve_main.

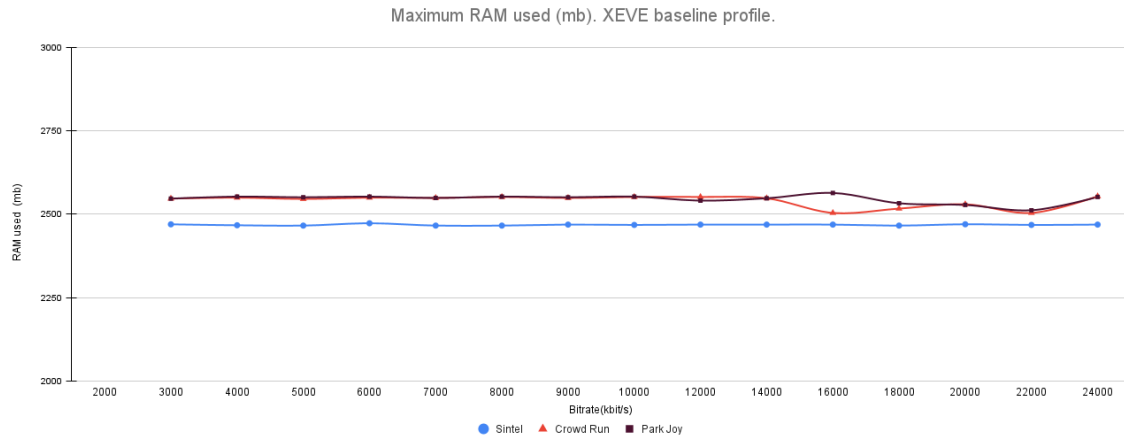


Figure 18: *Maximum RAM used encoding selected sequences using baseline profile of xeve.*
Chart created by author.

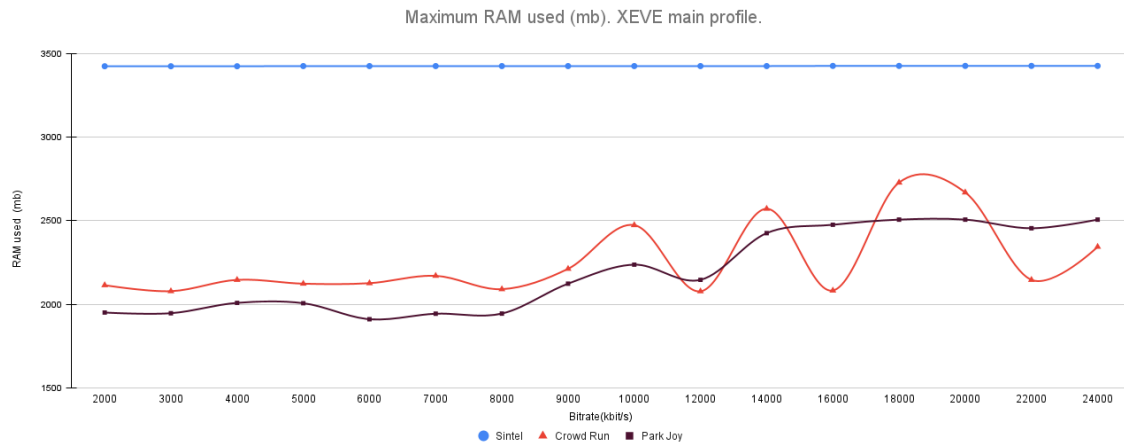


Figure 19: *Maximum RAM used encoding selected sequences using baseline profile of xeve.*
Chart created by author.

Conclusion

Based on the tests performed, the main profile of EVC consistently placed in the upper tier, often within few VMAF points or few dB's of VVC and ahead of other codecs like AV1, VP9 and HEVC, especially at low to midrange bitrates. The baseline profile, though often behind alternatives and struggling at lower bitrates, performs efficiently given its simpler toolset, and achieved good VMAF scoring at fraction of CPU time and RAM usage. Encoding complexity measurements indicate EVC moderate memory demands both for main and baseline profile.

However, despite the facts mentioned above, competing codecs already enjoy broad software and hardware support and well-understood licensing, leaving little room for a new format. For video streaming industry leaders, EVC's Baseline profile would deliver only modest gains in a few instances, while the Main profile, although more efficient, still requires patent licenses, undermining its intended simplicity. Additionally, there are no news regarding the royalty policy of the codec, making it unlikely to be adopted in the near future. The development ecosystem is also sparse: aside from open-source XEVE encoder/XEVD decoder efforts, few encoding or playback implementations exist in popular frameworks, and with only 3 major contributors, Samsung, Huawei, and Qualcomm, there is little progress made in terms of development. Without a clear adoption driver, it is unlikely that the industry will adopt EVC in the future [22, 23].

Of course, this work has limitations. The tests were performed using the default set of tools from the main profile and using fixed amount of CPU threads. Future research regarding EVC could attempt to test the codec on a broader range of video sequencing, perform tests on fewer, or, should codec gain such an option, more CPU threads, and experiment with switchable tools of the main profile. Experiments with human viewers could also be performed to reveal whether the small metric differences are perceptible to users.

References

- [1] Abhinav Rituraj. *AV1 Codec - Complete Guide for Video Application Devs*. Mar. 2024. (Visited on Mar. 30, 2025).
- [2] Alessandro Artusi, Andrea Basso, Marina Bosi, Sergio Canazza, Leonardo Chiariglione, Miran Choi, Fabiano Columbano, Mert Burkay Çöteli, Nadir Dalla Pozza, Roberto Dini, Michelangelo Guarise, Hüseyin Hacıhabiboğlu, Roberto Iacoviello, Chuanmin Jia, Jisu Kang, Panos Kudumakis, Valeria Lazzaroli, Marco Mazzaglia, Guido Perboli, Niccolò Pretto, Paolo Ribeca, Mariangela Rosano and Mark Seligman. ‘Video for Humans and Machines’. In: *Towards Pervasive and Trustworthy Artificial Intelligence*. Dec. 2021, pp. 49–52. (Visited on Mar. 16, 2025).
- [3] Supavadee Aramvith and Ming-Ting Sun. ‘MPEG-1 AND MPEG-2 Video Standards’. In: (1999).
- [4] Bruno Zagorščak. *Will HEVC Reduce Cost in the Video Business? - Neuralab*. Jan. 2016. (Visited on Mar. 22, 2025).
- [5] K. Choi, M. W. Park, K. P. Choi, J. Park, J. Chen, Y.-K. Wang, R. Chernyak, S. Ikonin, D. Rusanovskyy, W.-J. Chien, V. Seregin and M. Karczewicz. ‘MPEG-5: Essential Video Coding Standard’. In: *Applications of Digital Image Processing XLII*. Vol. 11137. SPIE, Sept. 2019, pp. 250–258. DOI: [10.1117/12.2530429](https://doi.org/10.1117/12.2530429). (Visited on May 11, 2025).
- [6] *Cisco Data Trends*. 2023. (Visited on May 11, 2025).
- [7] CSI. *MPEG-5 EVC Gets Final Approval*. May 2020. (Visited on Mar. 15, 2025).
- [8] *Decoding the Video Codec Wars: H.264, HEVC, and AV1 Compared for Streaming*. Aug. 2024. (Visited on Mar. 22, 2025).
- [9] Dmitriy Teplyakov. *A Review of the VVC Codec* \textbar *Elecard: Video Compression Guru*. Apr. 2021. (Visited on Apr. 12, 2025).
- [10] Mohammed Ghanbari. *Standard Codecs: Image Compression to Advanced Video Coding*. IET, 2003. ISBN: 978-0-85296-710-2. (Visited on May 10, 2025).
- [11] Diego Gibellino. *Introducing MPEG-5*. Mar. 2019. (Visited on Apr. 13, 2025).
- [12] Liwei Guo, Vivian Li, Jeff Watts, Julie Beckley and Venkatesh Selvaraj. *Netflix Now Streaming AV1 on Android*. Feb. 2020. (Visited on Mar. 30, 2025).
- [13] Philipp Habermann, Chi Ching Chi, Mauricio Alvarez-Mesa and Ben Juurlink. ‘Improved Wavefront Parallel Processing for HEVC Decoding’. In: (2017).

-
- [14] David N. Hein and Harry W. Jones Jr.
‘Conditional Replenishment Using Motion Prediction’.
In: *Applications of Digital Image Processing III*. Vol. 0207. SPIE, Dec. 1979,
pp. 268–277. DOI: [10.1117/12.958254](https://doi.org/10.1117/12.958254). (Visited on May 10, 2025).
 - [15] Christian Helmrich. *Ecodis :: Video Codecs*. Web-Blog. July 2024.
(Visited on Mar. 15, 2025).
 - [16] Yu-Wen Huang, Chih-Wei Hsu, Ching-Yeh Chen, Tzu-Der Chuang, Shih-Ta Hsiang,
Chun-Chia Chen, Man-Shu Chiang, Chen-Yen Lai, Chia-Ming Tsai, Yu-Chi Su,
Zhi-Yi Lin, Yu-Ling Hsiao, Olena Chubach, Yu-Cheng Lin and Shaw-Min Lei.
‘A VVC Proposal With Quaternary Tree Plus Binary-Ternary Tree Coding Block
Structure and Advanced Coding Techniques’. In: *IEEE Transactions on Circuits and
Systems for Video Technology* 30.5 (May 2020), pp. 1311–1325.
ISSN: 1051-8215, 1558-2205. DOI: [10.1109/TCSVT.2019.2945048](https://doi.org/10.1109/TCSVT.2019.2945048).
(Visited on Apr. 11, 2025).
 - [17] Iain Richardson. *HEVC: An Introduction to High Efficiency Coding*. 2024.
(Visited on Mar. 30, 2025).
 - [18] *ISO/IEC 23094-1:2020*. Oct. 2020. (Visited on May 10, 2025).
 - [19] *ITU-T Recommendation Database*. 2017. (Visited on Mar. 22, 2025).
 - [20] Marco Jacobs and Jonah Probell. ‘A Brief History of Video Coding’. In: (Jan. 2009).
 - [21] Jan Ozer. *Decoding the Landscape: Recent Developments in Video Codec Licensing*.
May 2024. (Visited on Mar. 30, 2025).
 - [22] Jan Ozer. *The State of the Video Codec Market 2025*. Mar. 2025.
(Visited on May 12, 2025).
 - [23] Jan Ozer. *The State of Video Codecs 2022*.
www.streamingmediaglobal.com/Articles/ReadArticle.aspx?ArticleID=152252.
Apr. 2022. (Visited on May 12, 2025).
 - [24] Jan Ozer. *What Is VVC?* Nov. 2019. (Visited on Apr. 12, 2025).
 - [25] Jun Xin, A. Vetro, S. Sekiguchi and K. Sugimoto. ‘MPEG-2 to H.264/AVC
Transcoding for Efficient Storage of Broadcast Video Bitstreams’. In: *2006 Digest of
Technical Papers International Conference on Consumer Electronics*.
Las Vegas, NV, USA: IEEE, 2006, pp. 417–418. ISBN: 978-0-7803-9459-9.
DOI: [10.1109/ICCE.2006.1598488](https://doi.org/10.1109/ICCE.2006.1598488). (Visited on Mar. 22, 2025).
 - [26] Ken McCann. *ITU Workshop on “The Future of Media”*. Oct. 2019.
(Visited on May 10, 2025).
 - [27] Tsahi Levent-Levi.
AV1 Video Codec Carries Clout with Royalty-Free Status \textbar TechTarget.
Apr. 2018. (Visited on Mar. 16, 2025).
 - [28] Markus Hafellner.
State of Compression: Testing h.266/VVC vs h.265/HEVC - Bitmovin. Dec. 2020.
(Visited on Apr. 3, 2025).
-

-
- [29] *Microsoft, Google, Amazon, Others, Aim for Royalty-Free Video Codecs*.
<https://arstechnica.com/information-technology/2015/09/microsoft-google-amazon-others-aim-for-royalty-free-video-codecs/>. Sept. 2015. (Visited on May 10, 2025).
 - [30] *MPEG 131 – MPEG*. 2020. (Visited on Apr. 14, 2025).
 - [31] Jens-Rainer Ohm, Gary J. Sullivan, Heiko Schwarz, Thiow Keng Tan and Thomas Wiegand. ‘Comparison of the Coding Efficiency of Video Coding Standards—Including High Efficiency Video Coding (HEVC)’. In: *IEEE Transactions on Circuits and Systems for Video Technology* 22.12 (Dec. 2012), pp. 1669–1684. ISSN: 1051-8215, 1558-2205. DOI: [10.1109/TCSVT.2012.2221192](https://doi.org/10.1109/TCSVT.2012.2221192). (Visited on May 11, 2025).
 - [32] S. Okubo. ‘Reference Model Methodology-A Tool for the Collaborative Creation of Video Coding Standards’. In: *Proceedings of the IEEE* 83.2 (Feb. 1995), pp. 139–150. ISSN: 00189219. DOI: [10.1109/5.364470](https://doi.org/10.1109/5.364470). (Visited on Mar. 16, 2025).
 - [33] *Open-Source Project for MPEG-5 EVC (Essential Video Coding)*. July 2021. (Visited on Apr. 14, 2025).
 - [34] Jan Ozer. *Inside MPEG’s Ambitious Plan to Launch 3 Video Codecs in 2020*. Oct. 2019. (Visited on Mar. 15, 2025).
 - [35] Petr Hovorka. ‘Objektivní porovnání kvality video kódovacího formátu AV1’. PhD thesis. Vysoká škola ekonomická v Praze, Mar. 2020.
 - [36] Nikolay Ponomarenko, Flavia Silvestri, Karen Egiazarian, Marco Carli, Jaakko Astola and Vladimir Lukin. ‘On Between-Coefficient Contrast Masking of DCT Basis Functions’. In: (Jan. 2007).
 - [37] RGB Spectrum. *H.264 Profiles*. (Visited on Mar. 22, 2025).
 - [38] Romain Bouqueau.
A View on VP9 and AV1 Part 1: Specifications - GPAC Licensing.
<https://web.archive.org/web/20210711073657/https://www.gpac-licensing.com/2016/07/12/vp9-av1-bitstream-format/>. July 2021. (Visited on May 10, 2025).
 - [39] Jonatan Samuelsson, Kiho Choi, Jianle Chen and Dmytro Rusanovskyy. ‘MPEG-5 Part 1: Essential Video Coding’. In: *SMPTE Motion Imaging Journal* 129.7 (Aug. 2020), pp. 10–16. ISSN: 1545-0279, 2160-2492. DOI: [10.5594/JMI.2020.3001795](https://doi.org/10.5594/JMI.2020.3001795). (Visited on May 11, 2025).
 - [40] R. Schafer and T. Sikora. ‘Digital Video Coding Standards and Their Role in Video Communications’. In: *Proceedings of the IEEE* 83.6 (June 1995), pp. 907–924. ISSN: 00189219. DOI: [10.1109/5.387092](https://doi.org/10.1109/5.387092). (Visited on Mar. 16, 2025).
 - [41] H.R. Sheikh and A.C. Bovik. ‘Image Information and Visual Quality’. In: *IEEE Transactions on Image Processing* 15.2 (Feb. 2006), pp. 430–444. ISSN: 1941-0042. DOI: [10.1109/TIP.2005.859378](https://doi.org/10.1109/TIP.2005.859378). (Visited on May 11, 2025).
-

-
- [42] Gary J. Sullivan. ‘Overview of International Overview of International Video Coding Standards Video Coding Standards (Preceding H.264/AVC) (Preceding H.264/AVC)’. In: (July 2005).
- [43] *The History of Video File Formats Infographic — RealPlayer*. Apr. 2012. (Visited on Mar. 17, 2025).
- [44] Thomas Wiegand. *Video Coding Standards*. (Visited on May 10, 2025).
- [45] Tomáš Pydych. ‘An Overview and Comparison of the Versatile Video Coding Standard’. PhD thesis. Prague: Vysoká škola ekonomická v Praze, Dec. 2025.
- [46] Yubing Tong, Hubert Konik, Faouzi Alaya Cheikh and Alain Treméau. ‘Full Reference Image Quality Assessment Based on Saliency Map Analysis’. In: *Journal of Imaging Science and Technology* 54 (May 2010). DOI: [10.2352/J.ImagingSci.Technol.2010.54.3.030503](https://doi.org/10.2352/J.ImagingSci.Technol.2010.54.3.030503).
- [47] Russell Trafford-Jones. *MPEG-5 Essential Video Coding (EVC) Standard*. Feb. 2021. (Visited on May 10, 2025).
- [48] Miroslav Uhrina, Lukas Sevcik, Juraj Bienik and Lenka Smatanova. ‘Performance Comparison of VVC, AV1, HEVC, and AVC for High Resolutions’. In: *Electronics* 13.5 (Jan. 2024), p. 953. ISSN: 2079-9292. DOI: [10.3390/electronics13050953](https://doi.org/10.3390/electronics13050953). (Visited on Apr. 12, 2025).
- [49] *Video Developer Report 2019*. <https://go.bitmovin.com/video-developer-report-2019>. 2019. (Visited on May 10, 2025).
- [50] Krishna Rao Vijayanagar. *What Is Essential Video Coding (EVC) MPEG-5 Part-1?* Feb. 2021. (Visited on Mar. 15, 2025).
- [51] Z. Wang, E.P. Simoncelli and A.C. Bovik. ‘Multiscale Structural Similarity for Image Quality Assessment’. In: *The Thrity-Seventh Asilomar Conference on Signals, Systems & Computers, 2003*. Pacific Grove, CA, USA: IEEE, 2003, pp. 1398–1402. ISBN: 978-0-7803-8104-9. DOI: [10.1109/ACSSC.2003.1292216](https://doi.org/10.1109/ACSSC.2003.1292216). (Visited on May 10, 2025).
- [52] Shuai Zhao, Stephan Wenger and Youngkwon Lim. *RTP Payload Format for Essential Video Coding (EVC)*. Request for Comments RFC 9584. Internet Engineering Task Force, June 2024. DOI: [10.17487/RFC9584](https://doi.org/10.17487/RFC9584). (Visited on May 10, 2025).
- [53] Zhi Li, Anne Aaron, Ioannis Katsavounidis and Anush Moorthy. *Toward A Practical Perceptual Video Quality Metric*. <https://netflixtechblog.com/toward-a-practical-perceptual-video-quality-metric-653f208b9652>. Apr. 2017. (Visited on May 11, 2025).
-

Appendices

A. Quality testing data

Table 5: *VMAF values for XEVE baseline profile.*

Bitrate (kbit/s)	Sintel	Crowd Run	Park Joy
2000	N/A	N/A	N/A
3000	88.564 320	47.788 169	48.230 466
4000	91.370 377	47.788 169	49.181 369
5000	93.208 517	48.272 074	49.669 257
6000	94.263 636	51.224 431	52.236 290
7000	95.208 187	54.310 353	57.435 553
8000	96.876 552	57.845 777	60.935 090
9000	96.252 414	61.217 725	63.210 759
10 000	96.453 692	64.017 866	64.294 668
12 000	97.438 148	69.393 910	68.542 548
14 000	98.443 787	73.598 312	69.784 123
16 000	98.730 062	77.015 289	72.625 657
18 000	98.892 983	79.949 874	74.123 701
20 000	98.899 163	82.229 144	73.882 767
22 000	99.408 834	84.679 514	76.772 275
24 000	99.382 179	86.513 745	76.423 244

Table 6: *VMAF values for XEVE main profile.*

Bitrate (kbit/s)	Sintel	Crowd Run	Park Joy
2000	93.009 003	51.624 191	50.605 648
3000	93.556 021	51.684 349	52.355 265
4000	95.251 716	55.700 807	53.159 452
5000	95.778 921	60.703 536	56.265 789
6000	96.173 730	64.704 434	60.394 682
7000	96.522 103	68.706 882	63.620 363
8000	96.613 282	71.864 824	67.344 311
9000	96.899 630	74.913 928	69.152 830
10 000	97.075 893	77.716 125	71.587 162
12 000	97.523 192	81.231 983	76.019 825
14 000	98.785 556	84.734 495	78.796 428
16 000	99.075 054	87.389 970	81.185 029
18 000	99.338 979	89.703 552	83.774 166
20 000	99.264 195	90.547 967	85.345 895
22 000	99.459 218	92.087 043	87.353 448
24 000	99.477 379	93.322 036	90.437 893

Table 7: *MS-SSIM values for XEVE base profile.*

Bitrate (kbit/s)	Sintel	Crowd Run	Park Joy
2000	N/A	N/A	N/A
3000	0.964 082	0.821 238	0.807 464
4000	0.969 510	0.821 238	0.815 683
5000	0.973 956	0.825 308	0.818 860
6000	0.976 743	0.841 924	0.833 535
7000	0.979 188	0.858 016	0.861 436
8000	0.983 697	0.873 064	0.879 803
9000	0.982 486	0.886 940	0.889 587
10 000	0.983 404	0.896 602	0.892 724
12 000	0.986 127	0.912 966	0.907 971
14 000	0.989 207	0.924 421	0.911 201
16 000	0.990 519	0.932 920	0.919 524
18 000	0.991 625	0.939 397	0.924 457
20 000	0.992 192	0.944 337	0.923 373
22 000	0.994 460	0.949 336	0.931 178
24 000	0.994 962	0.952 704	0.930 607

Table 8: *MS-SSIM values for XEVE main profile.*

Bitrate (kbit/s)	Sintel	Crowd Run	Park Joy
2000	0.972 590	0.838 265	0.825 239
3000	0.974 327	0.838 174	0.824 253
4000	0.976 509	0.858 045	0.831 206
5000	0.979 012	0.881 422	0.854 125
6000	0.982 627	0.897 063	0.868 843
7000	0.983 686	0.909 427	0.879 809
8000	0.983 890	0.918 731	0.896 958
9000	0.984 642	0.927 415	0.905 137
10 000	0.985 913	0.934 433	0.911 631
12 000	0.987 863	0.942 769	0.926 893
14 000	0.992 248	0.950 342	0.932 850
16 000	0.993 972	0.955 821	0.941 524
18 000	0.995 877	0.959 879	0.945 282
20 000	0.995 902	0.961 327	0.947 352
22 000	0.996 882	0.964 440	0.953 371
24 000	0.997 243	0.966 628	0.961 642

Table 9: *PSNR-HVS-M values for XEVE base profile.*

Bitrate (kbit/s)	Sintel	Crowd Run	Park Joy
2000	N/A	N/A	N/A
3000	36.222 977	22.282 621	21.086 632
4000	37.608 204	22.282 621	21.353 456
5000	38.947 578	22.375 147	21.426 233
6000	39.649 334	22.915 674	21.928 555
7000	40.682 499	23.479 658	22.905 819
8000	41.502 575	24.128 935	23.544 950
9000	42.002 224	24.752 821	23.990 419
10 000	42.529 110	25.284 504	24.233 290
12 000	43.396 206	26.306 892	25.095 991
14 000	44.574 829	27.160 870	25.475 035
16 000	45.481 907	27.896 721	26.101 542
18 000	46.362 349	28.567 110	26.499 687
20 000	46.931 885	29.113 653	26.592 360
22 000	47.773 708	29.744 036	27.238 928
24 000	48.156 025	30.231 836	27.267 138

Table 10: *PSNR-HVS-M values for XEVE main profile.*

Bitrate (kbit/s)	Sintel	Crowd Run	Park Joy
2000	37.084 290	23.088 964	21.723 597
3000	38.682 312	23.090 462	21.884 313
4000	40.057 812	23.892 008	22.268 229
5000	41.262 409	24.917 379	22.962 432
6000	41.845 463	25.757 481	23.651 480
7000	42.362 890	26.517 597	24.567 241
8000	42.917 164	27.181 486	25.038 914
9000	43.236 855	27.839 474	25.531 272
10 000	43.959 465	28.464 306	25.930 038
12 000	45.138 355	29.311 853	26.483 951
14 000	46.486 881	30.216 183	27.575 623
16 000	47.686 398	30.970 646	28.163 980
18 000	48.712 814	31.664 263	28.866 457
20 000	49.203 556	31.932 690	29.235 609
22 000	49.935 299	32.490 669	29.885 042
24 000	49.935 299	32.964 851	30.968 272

B. Time and memory usage

Table 11: *Encoding time for sequences, in seconds. XEVE base profile.*

Bitrate (kbit/s)	Sintel	Crowd Run	Park Joy
2000	N/A	N/A	N/A
3000	517	820	838
4000	535	820	842
5000	551	830	839
6000	562	851	878
7000	573	861	889
8000	588	879	883
9000	596	889	886
10 000	601	915	898
12 000	618	935	912
14 000	634	965	923
16 000	644	973	946
18 000	655	989	946
20 000	666	1001	946
22 000	682	1020	974
24 000	692	1035	966

Table 12: *Encoding time for sequences, in seconds. XEVE main profile.*

Bitrate (kbit/s)	Sintel	Crowd Run	Park Joy
2000	7834	12 081	12 532
3000	8526	12 291	12 954
4000	10 580	16 452	14 478
5000	11 854	16 491	14 875
6000	13 114	24 851	18 271
7000	13 988	23 278	20 863
8000	14 952	26 018	22 184
9000	15 052	28 468	23 674
10 000	16 671	29 341	24 697
12 000	18 654	33 029	27 634
14 000	18 984	37 058	30 017
16 000	22 760	39 446	32 684
18 000	24 660	43 077	34 487
20 000	25 243	44 479	37 346
22 000	26 526	43 975	38 854
24 000	32 092	48 887	40 633

Table 13: *Memory usage during encoding, in kilobytes. XEVE base profile.*

Bitrate (kbit/s)	Sintel	Crowd Run	Park Joy
2000	N/A	N/A	N/A
3000	2 469 820	2 547 792	2 547 072
4000	2 467 348	2 550 732	2 553 800
5000	2 466 140	2 546 712	2 551 628
6000	2 473 564	2 550 916	2 553 404
7000	2 466 820	2 549 096	2 549 932
8000	2 466 700	2 552 964	2 555 372
9000	2 469 976	2 549 248	2 551 232
10 000	2 468 580	2 552 932	2 553 108
12 000	2 469 240	2 552 104	2 549 116
14 000	2 469 216	2 548 096	2 548 664
16 000	2 469 612	2 504 400	2 564 008
18 000	2 466 640	2 517 800	2 533 580
20 000	2 470 216	2 530 212	2 528 060
22 000	2 468 908	2 504 916	2 512 980
24 000	2 469 288	2 554 220	2 552 504

Table 14: *Memory usage during encoding, in kilobytes. XEVE main profile.*

Bitrate (kbit/s)	Sintel	Crowd Run	Park Joy
2000	3 426 688	2 111 476	1 950 864
3000	3 426 800	2 078 552	1 946 708
4000	3 426 944	2 146 800	2 008 392
5000	3 427 200	2 123 864	2 006 422
6000	3 427 200	2 126 800	1 910 480
7000	3 427 200	2 170 620	1 943 648
8000	3 427 200	2 090 988	1 944 644
9000	3 427 200	2 212 604	2 123 800
10 000	3 427 584	2 474 724	2 237 776
12 000	3 427 840	2 077 464	2 146 580
14 000	3 427 968	2 572 168	2 426 904
16 000	3 428 096	2 082 248	2 476 884
18 000	3 428 352	2 729 080	2 507 100
20 000	3 428 096	2 670 528	2 507 622
22 000	3 428 224	2 146 580	2 455 368
24 000	3 428 096	2 344 760	2 507 100